

Joint Embedding Predictive World Models

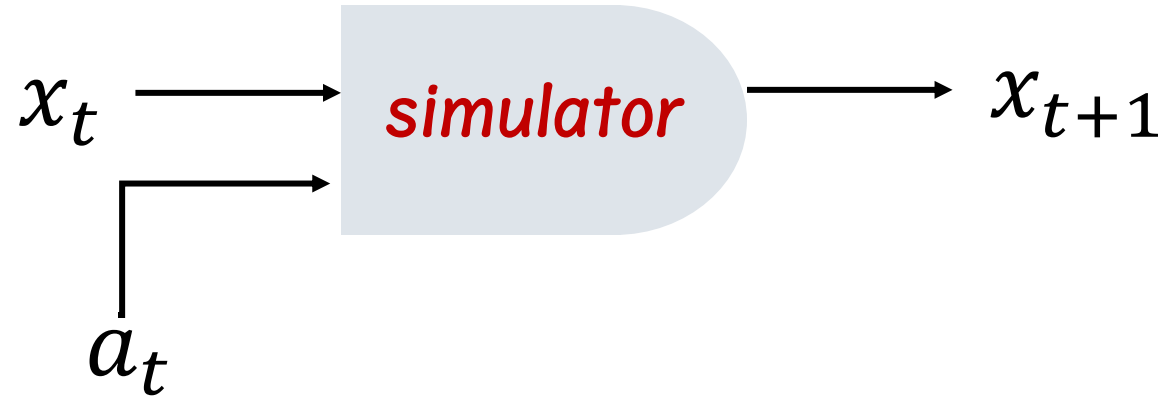
Heejeong Nam, Lucas Maes



Stanford CS25
United Transformer V6

1. *Concepts of JEPA and World Models*
2. *How to make a model understand object interaction*
3. *How to train JEPA end-to-end without collapse*

what is a world model



1. *Good State Representation*

what matters now

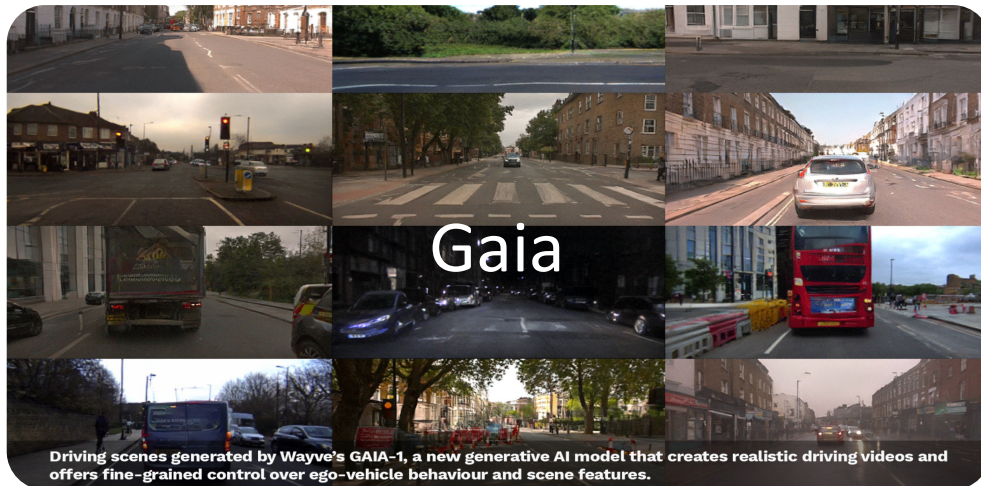
2. *Good Transition Model*

how the world evolves

3. *Good Dynamics Model*

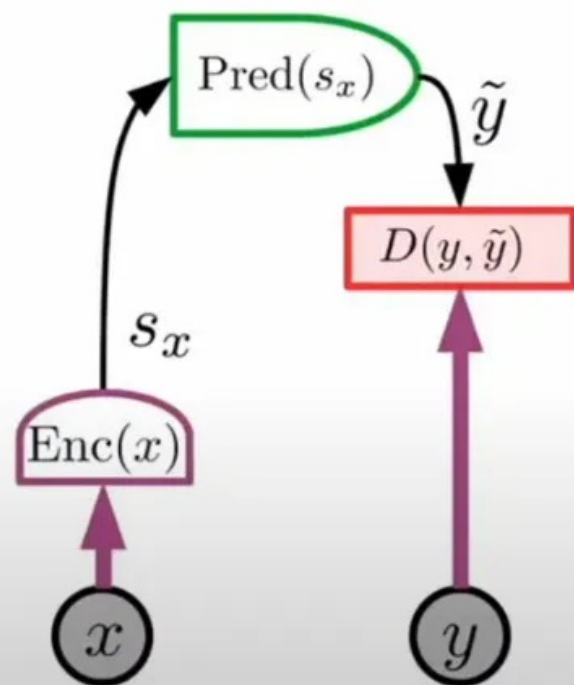
how the world responds to you

(We will not cover) Generative World Models

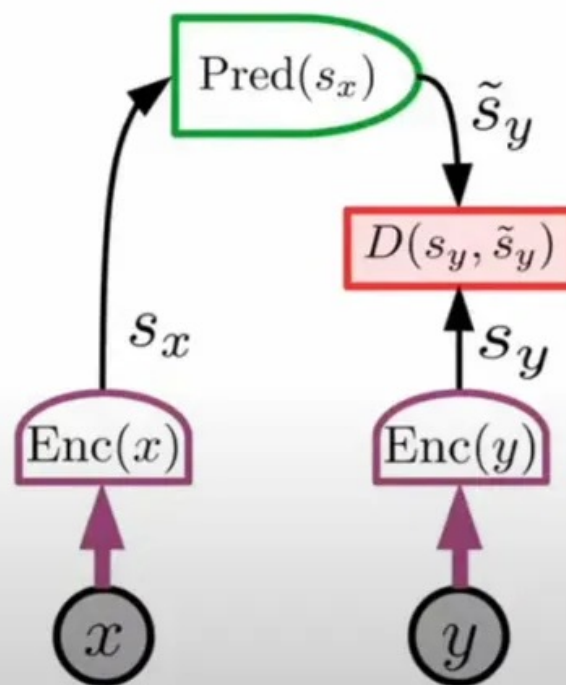


Architectures: Generative vs Joint Embedding

- ▶ **Generative:** predicts y (with all the details, including irrelevant ones)
- ▶ **Joint Embedding:** predicts an **abstract representation** of y

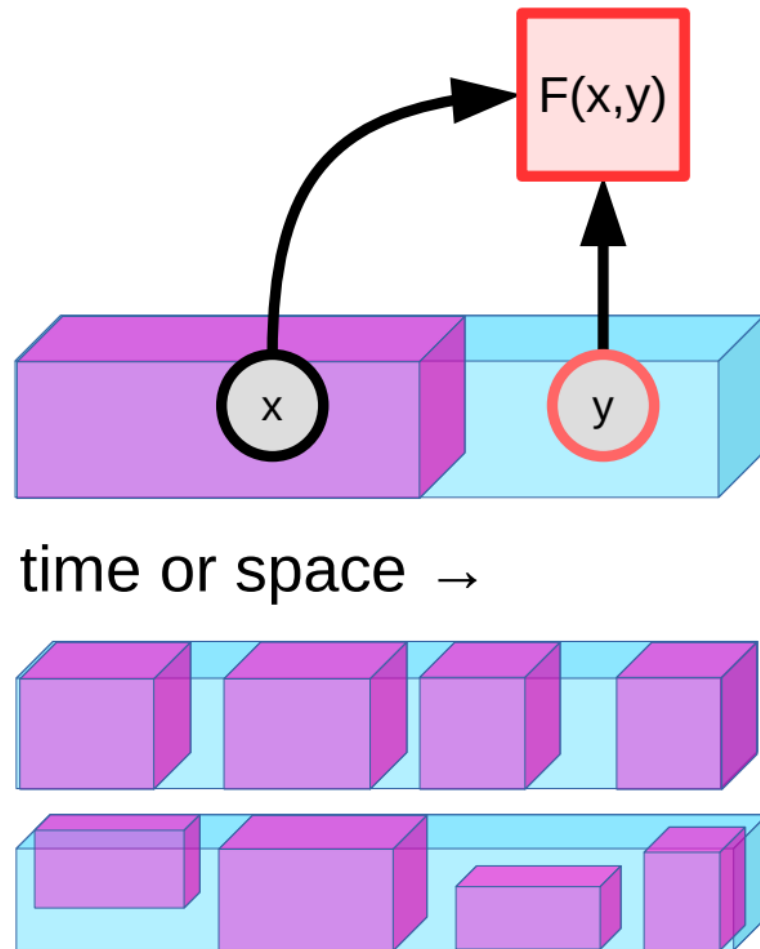


a) Generative Architecture
Examples: VAE, MAE...



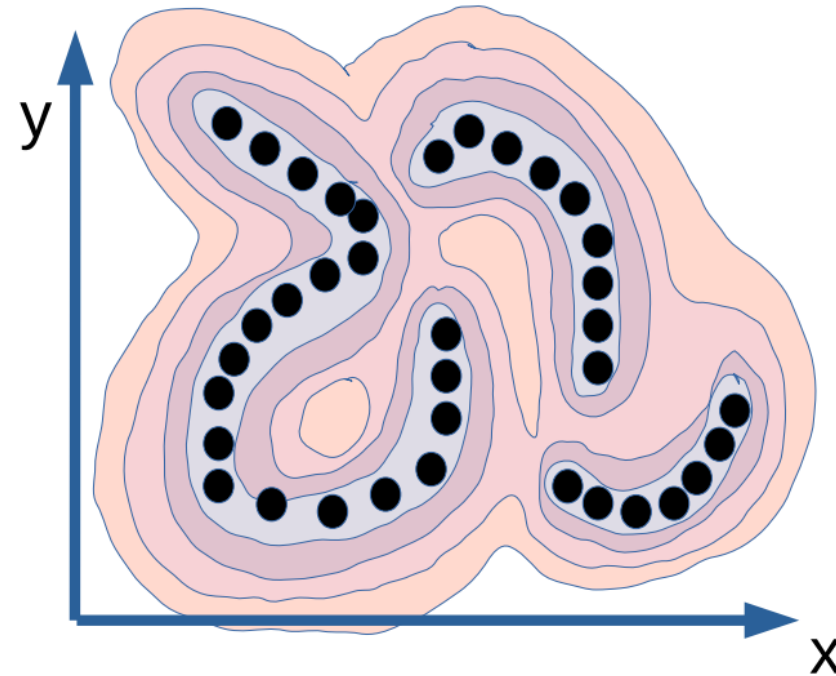
b) Joint Embedding Architecture

JEPA (LeCun, 2022)

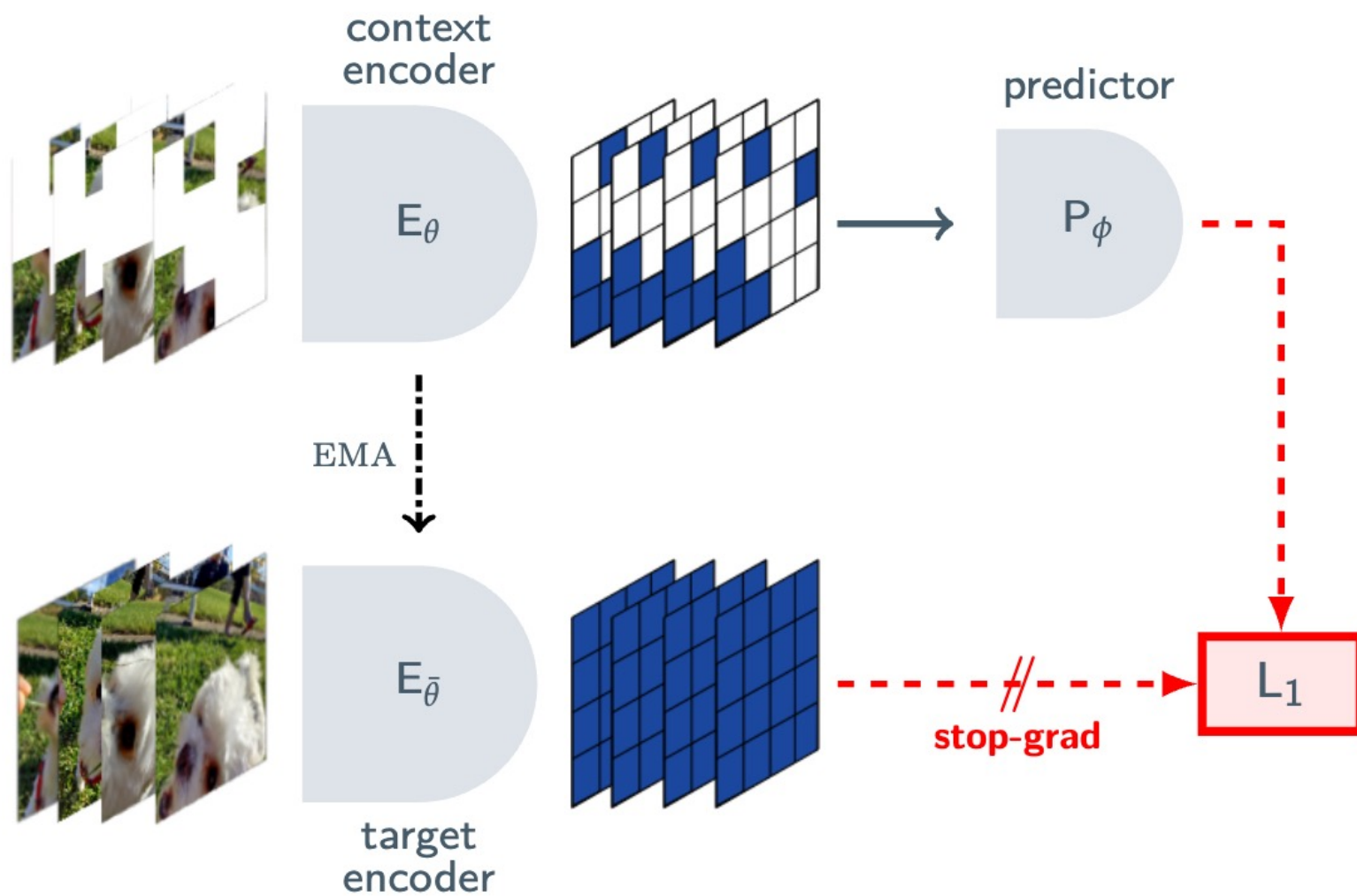


Energy
Function

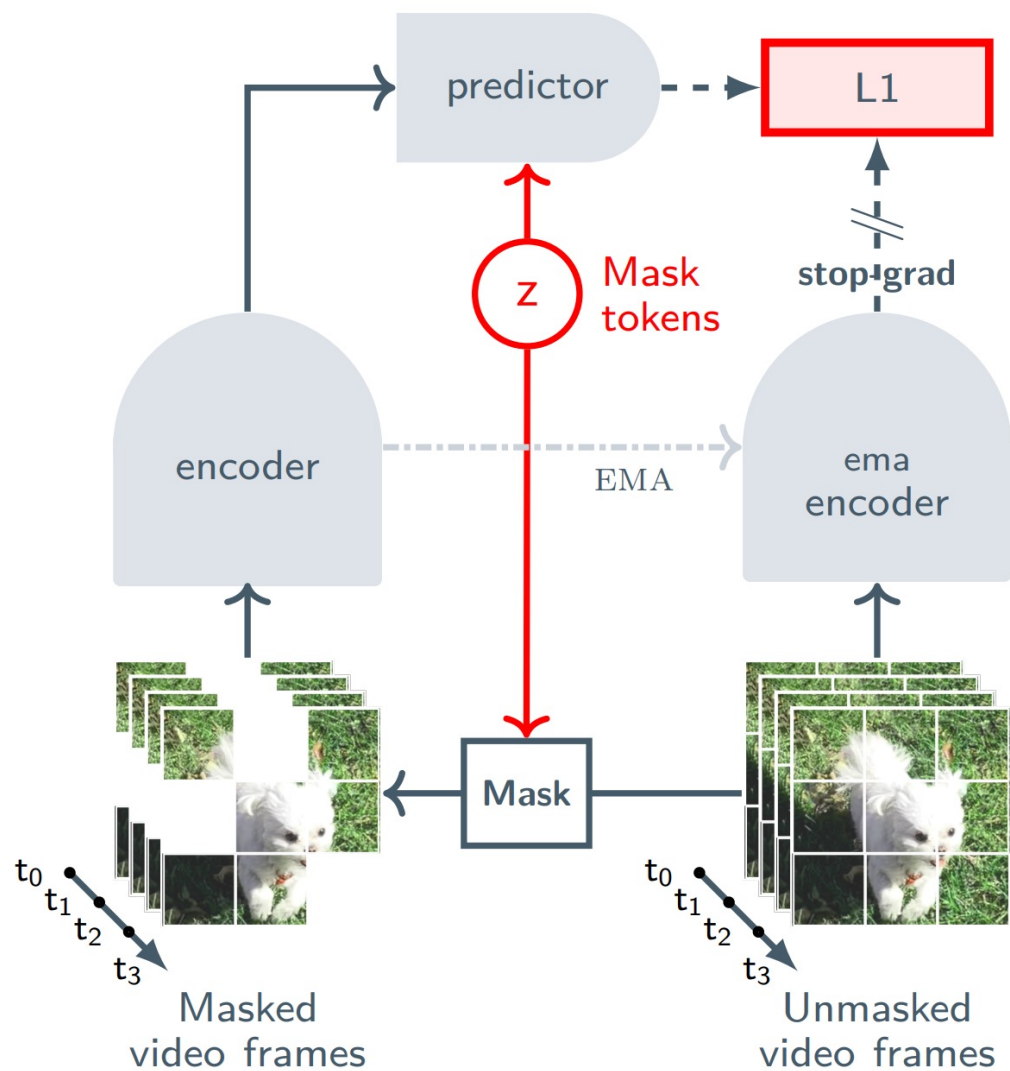
Energy Based Model



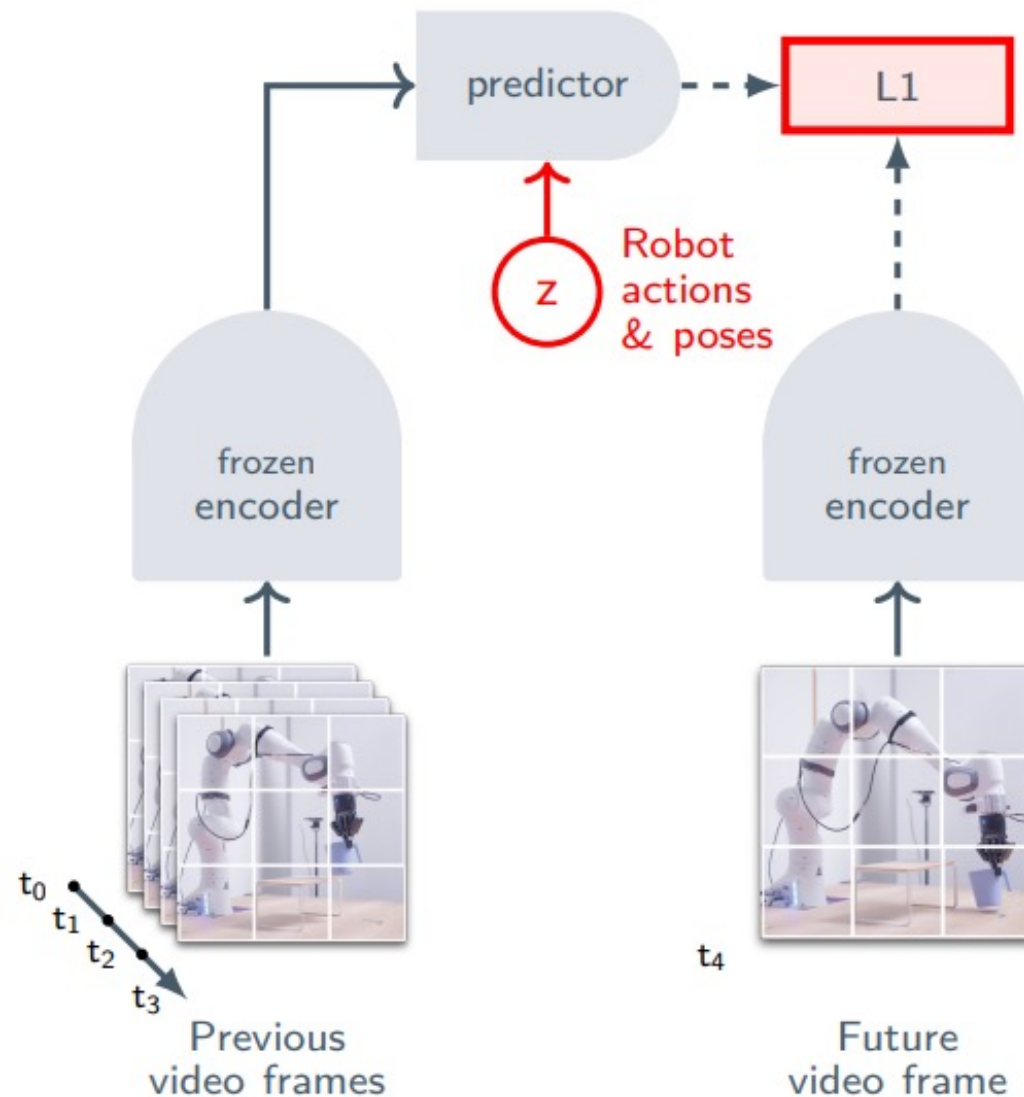
V-JEPA (Bardes et al., 2024)



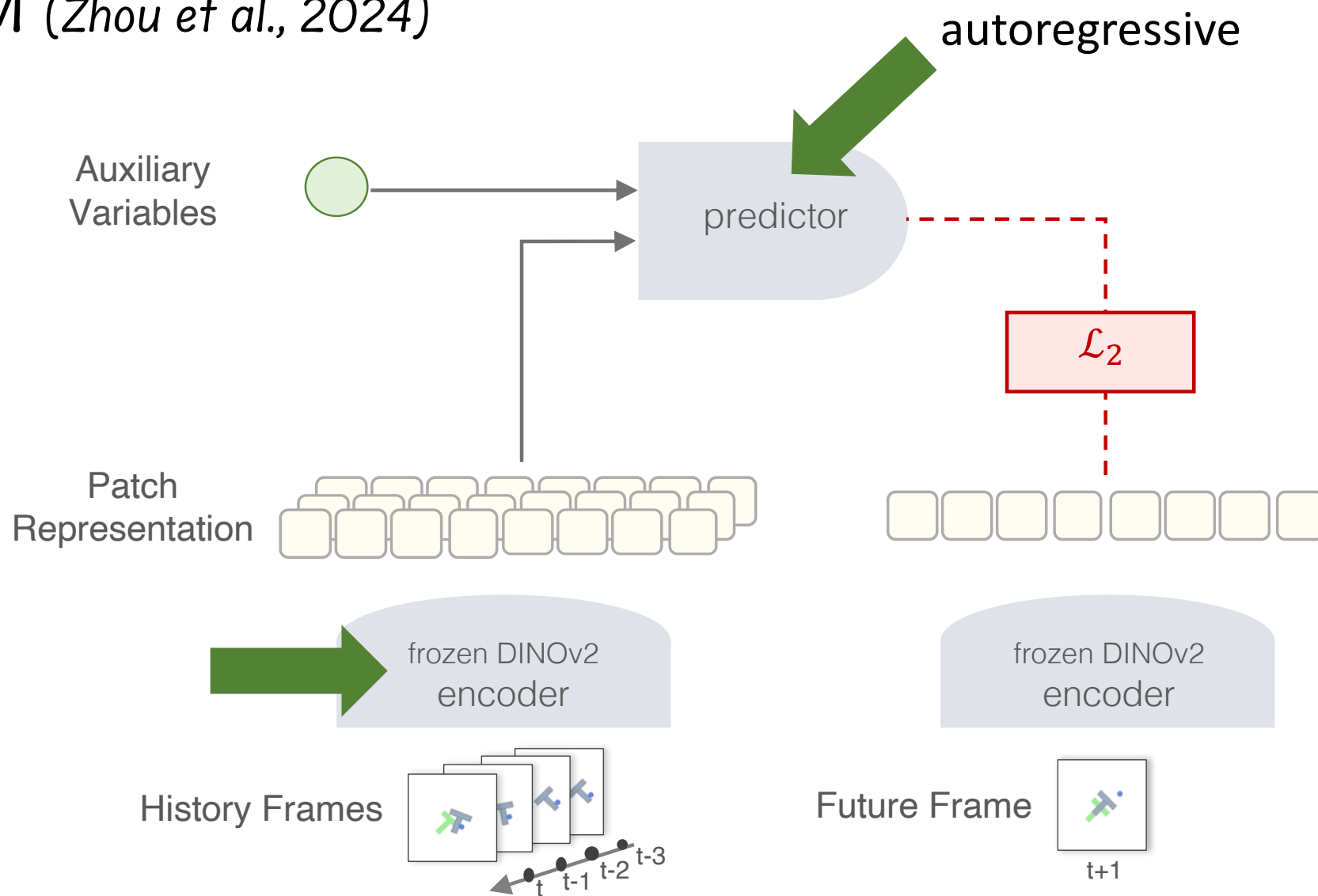
V-JEPA 2 (Assran et al., 2025)



Action Conditioned $\rightarrow$ World Model



DINO-WM (Zhou et al., 2024)



Causal-JEPA: Learning World Models through Object-Level Latent Interventions

Collaborators



Heejeong Nam



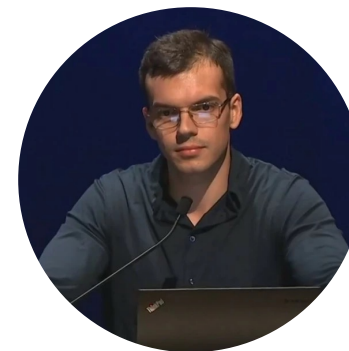
Quentin Le Lidec



Lucas Maes



Yann LeCun



Randall Balestriero



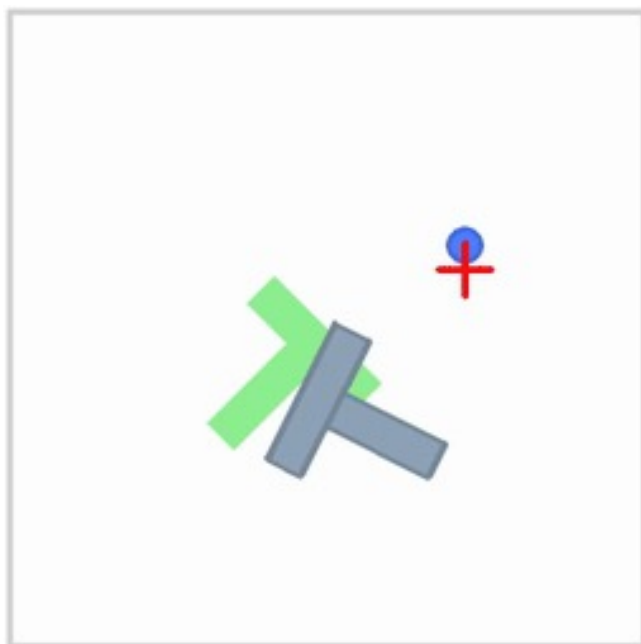
BROWN



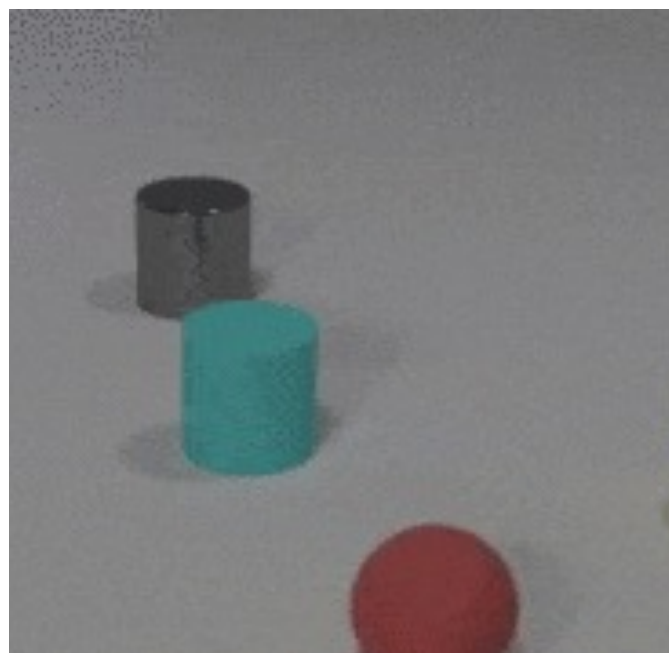
NYU



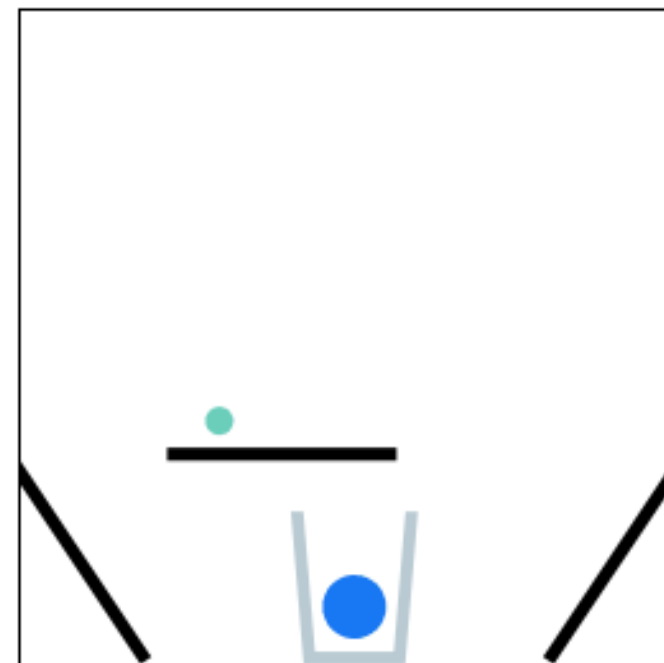
Mila



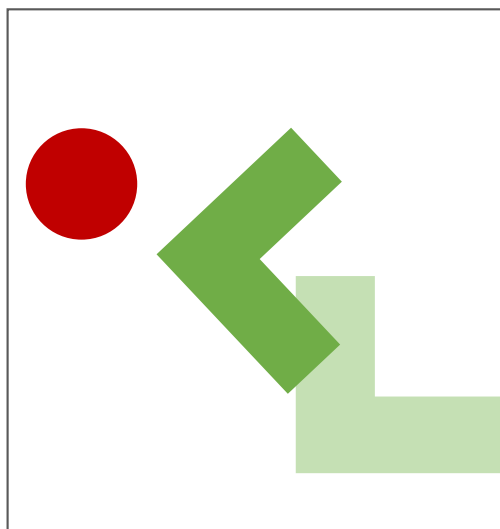
Push-T



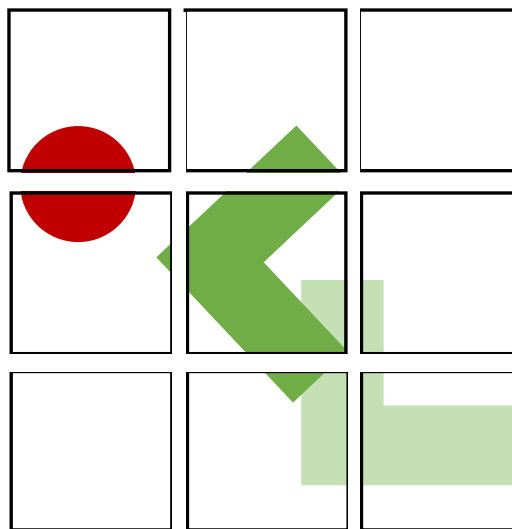
CLEVRER



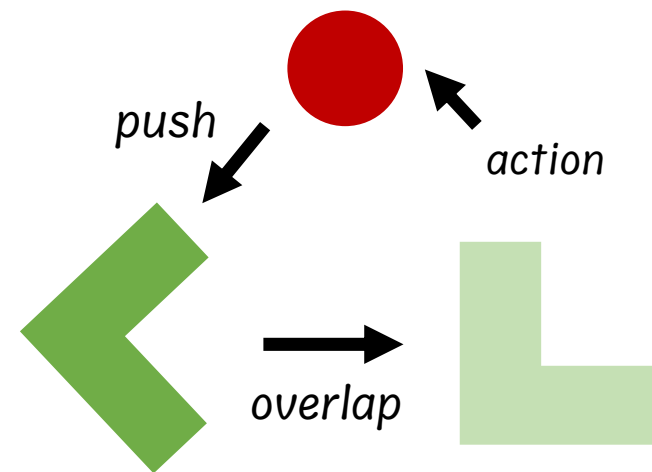
PHYRE



Observation

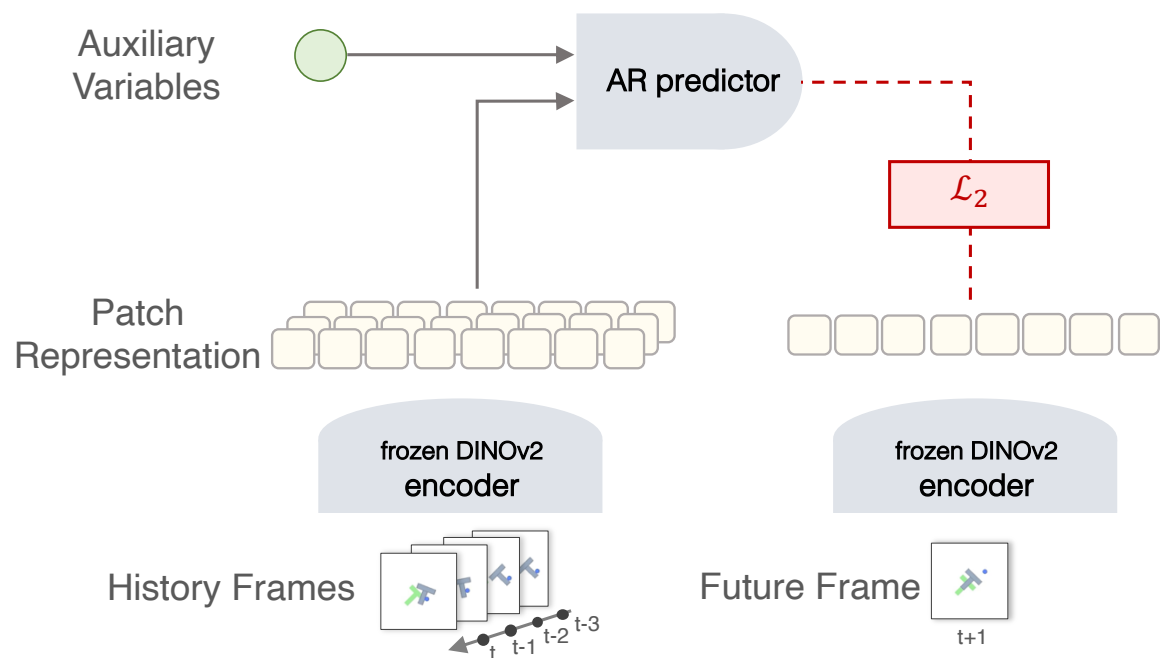


Patch-based Models

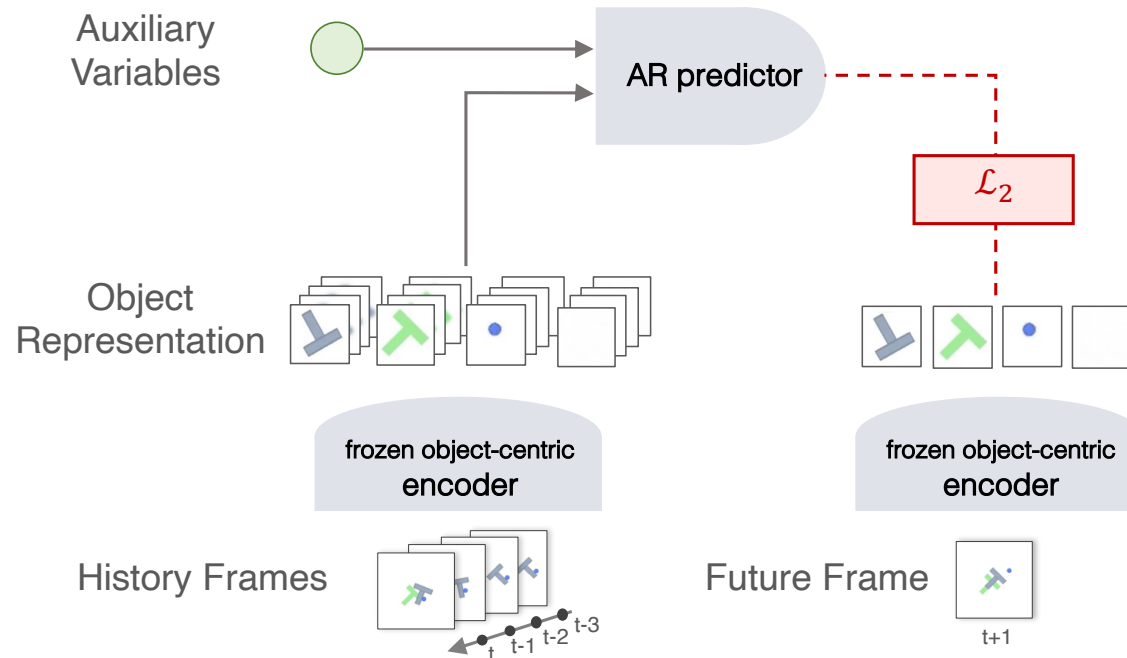


What is natural for us

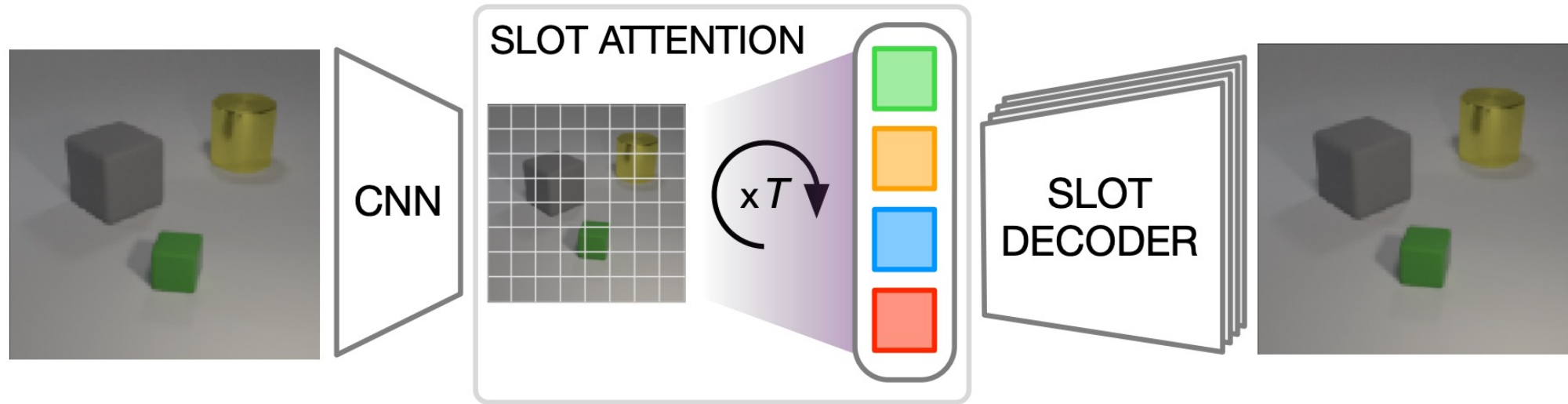
DINO-WM



OC-DINO-WM



Object Centric Learning

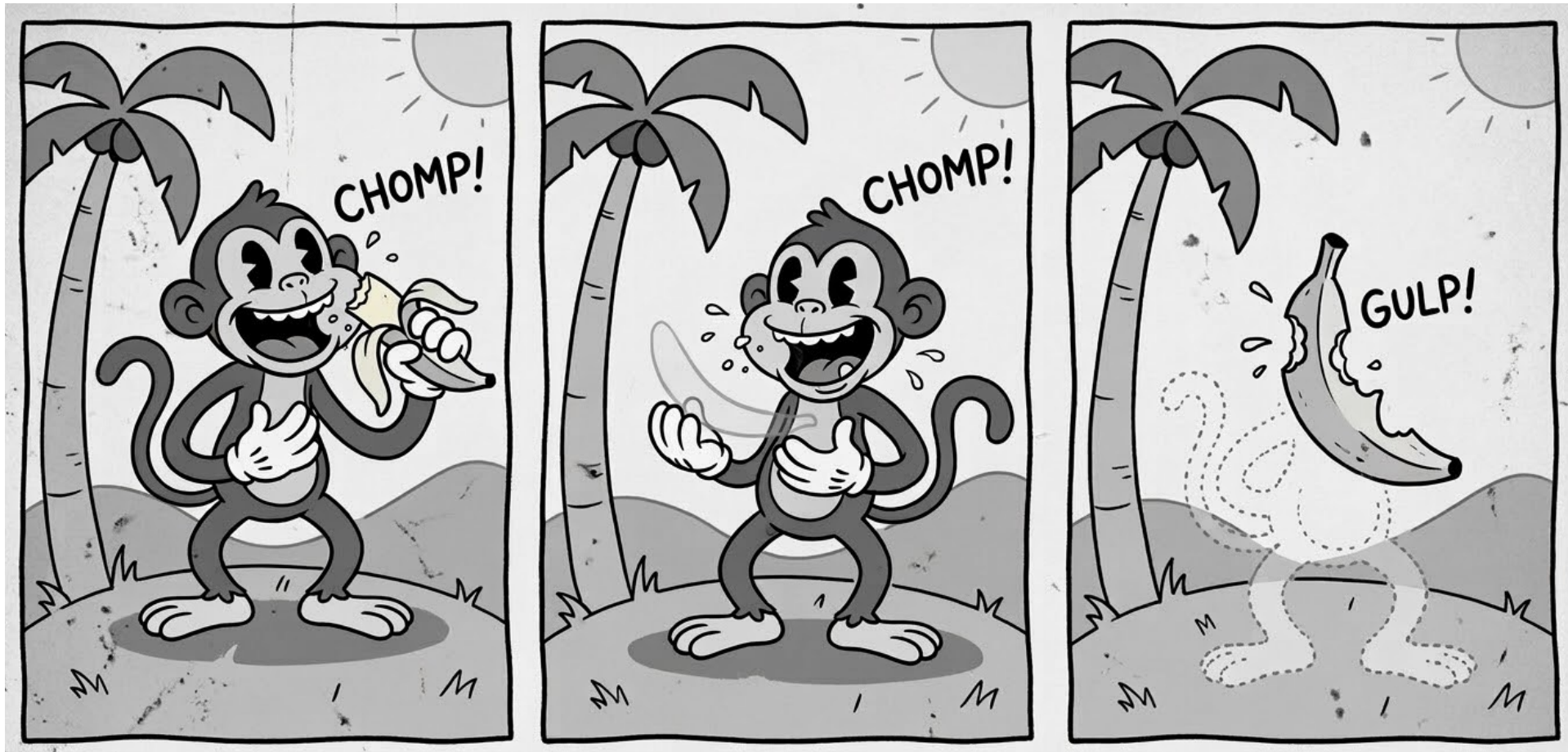


Algorithm 1 Slot Attention module. The input is a set of N vectors of dimension D_{inputs} which is mapped to a set of K slots of dimension D_{slots} . We initialize the slots by sampling their initial values as independent samples from a Gaussian distribution with shared, learnable parameters $\mu \in \mathbb{R}^{D_{\text{slots}}}$ and $\sigma \in \mathbb{R}^{D_{\text{slots}}}$. In our experiments we set the number of iterations to $T = 3$.

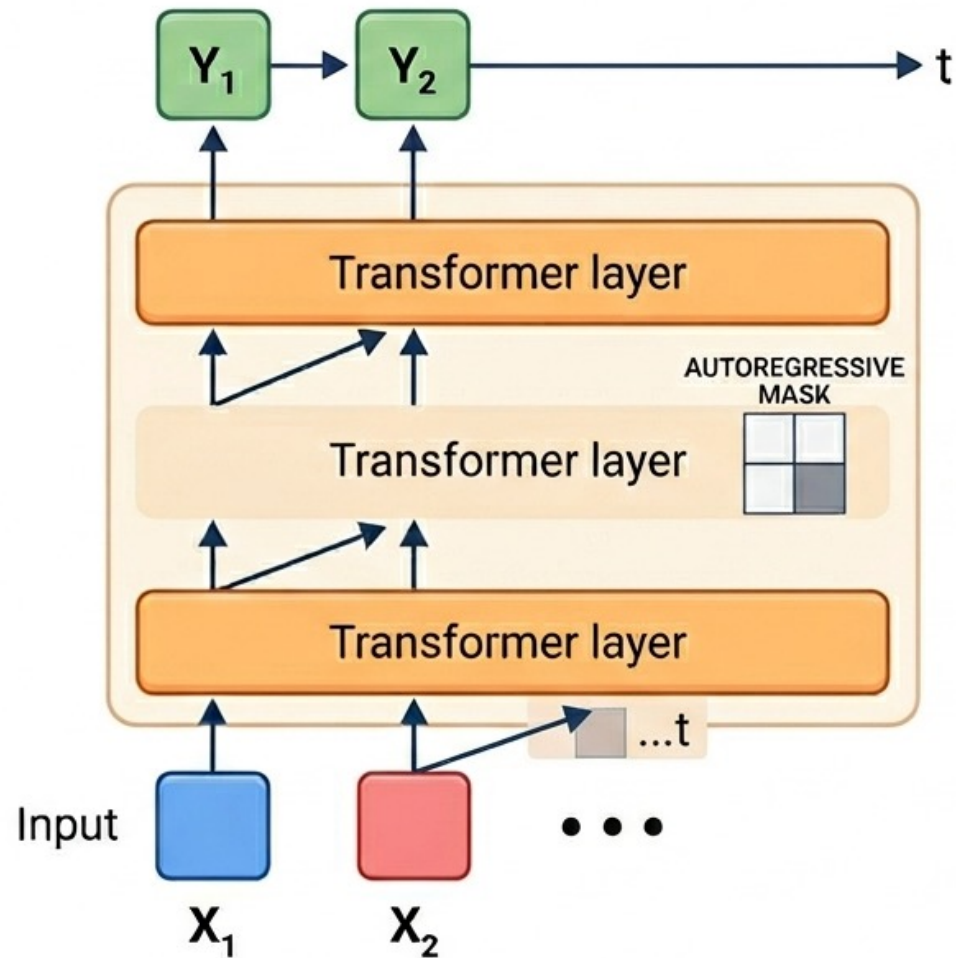
```

1: Input:  $\text{inputs} \in \mathbb{R}^{N \times D_{\text{inputs}}}$ ,  $\text{slots} \sim \mathcal{N}(\mu, \text{diag}(\sigma)) \in \mathbb{R}^{K \times D_{\text{slots}}}$ 
2: Layer params:  $k, q, v$ : linear projections for attention; GRU; MLP; LayerNorm(x3)
3:    $\text{inputs} = \text{LayerNorm}(\text{inputs})$ 
4:   for  $t = 0 \dots T$ 
5:      $\text{slots\_prev} = \text{slots}$ 
6:      $\text{slots} = \text{LayerNorm}(\text{slots})$ 
7:      $\text{attn} = \text{Softmax}(\frac{1}{\sqrt{D}} k(\text{inputs}) \cdot q(\text{slots})^T, \text{axis}='slots')$  # norm. over slots
8:      $\text{updates} = \text{WeightedMean}(\text{weights}=\text{attn} + \epsilon, \text{values}=v(\text{inputs}))$  # aggregate
9:      $\text{slots} = \text{GRU}(\text{state}=\text{slots\_prev}, \text{inputs}=\text{updates})$  # GRU update (per slot)
10:     $\text{slots} += \text{MLP}(\text{LayerNorm}(\text{slots}))$  # optional residual MLP (per slot)
11:   return  $\text{slots}$ 

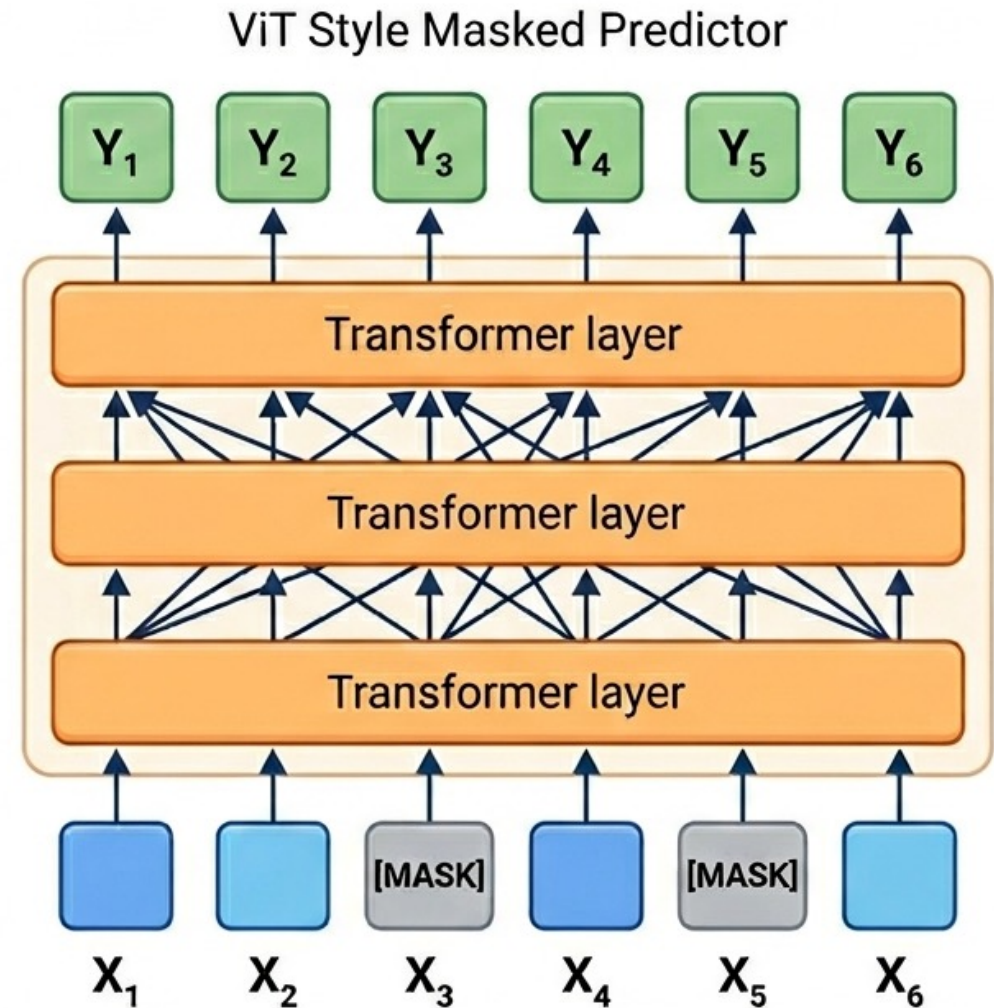
```

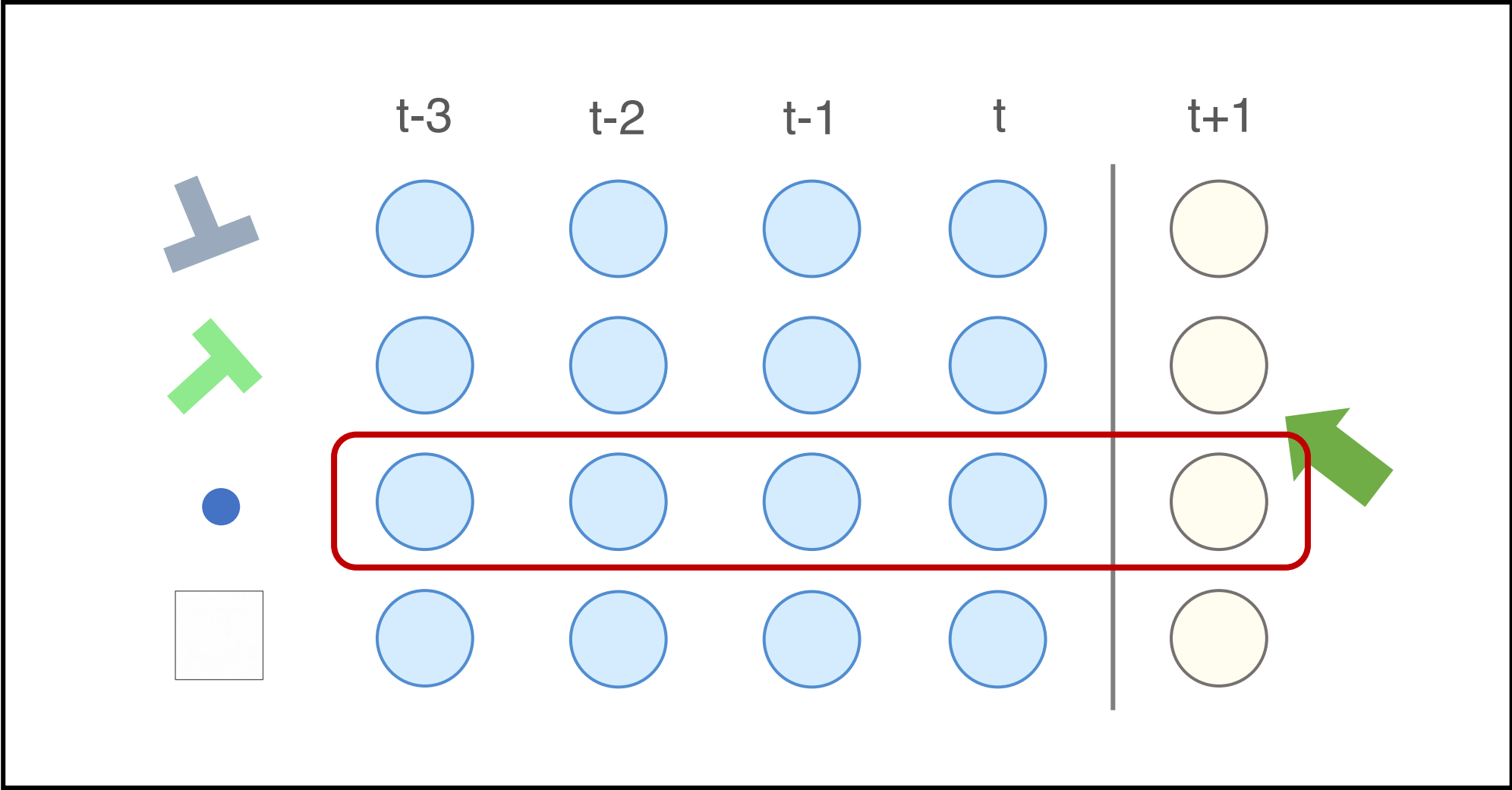


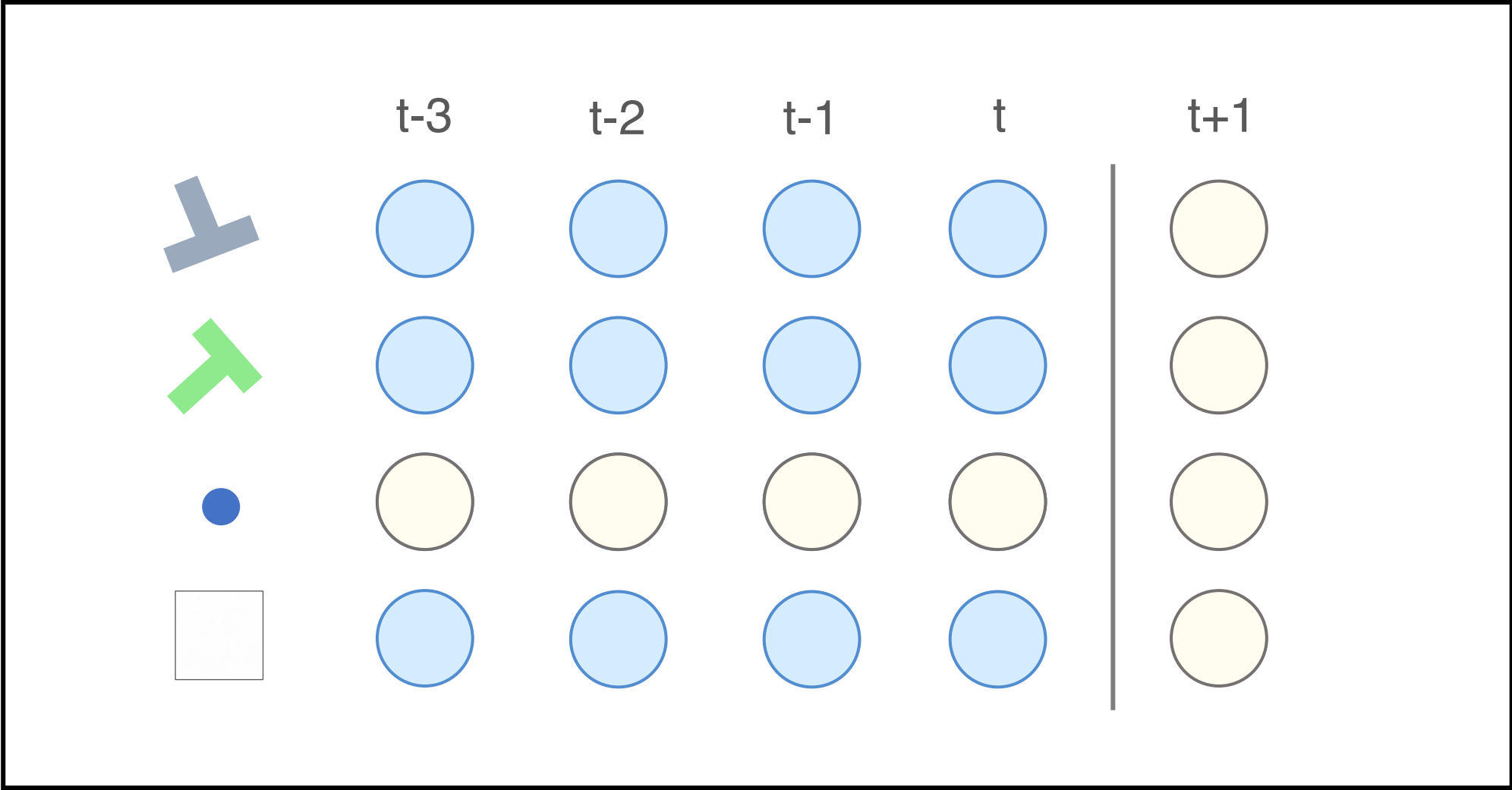
CAUSAL TRANSFORMER

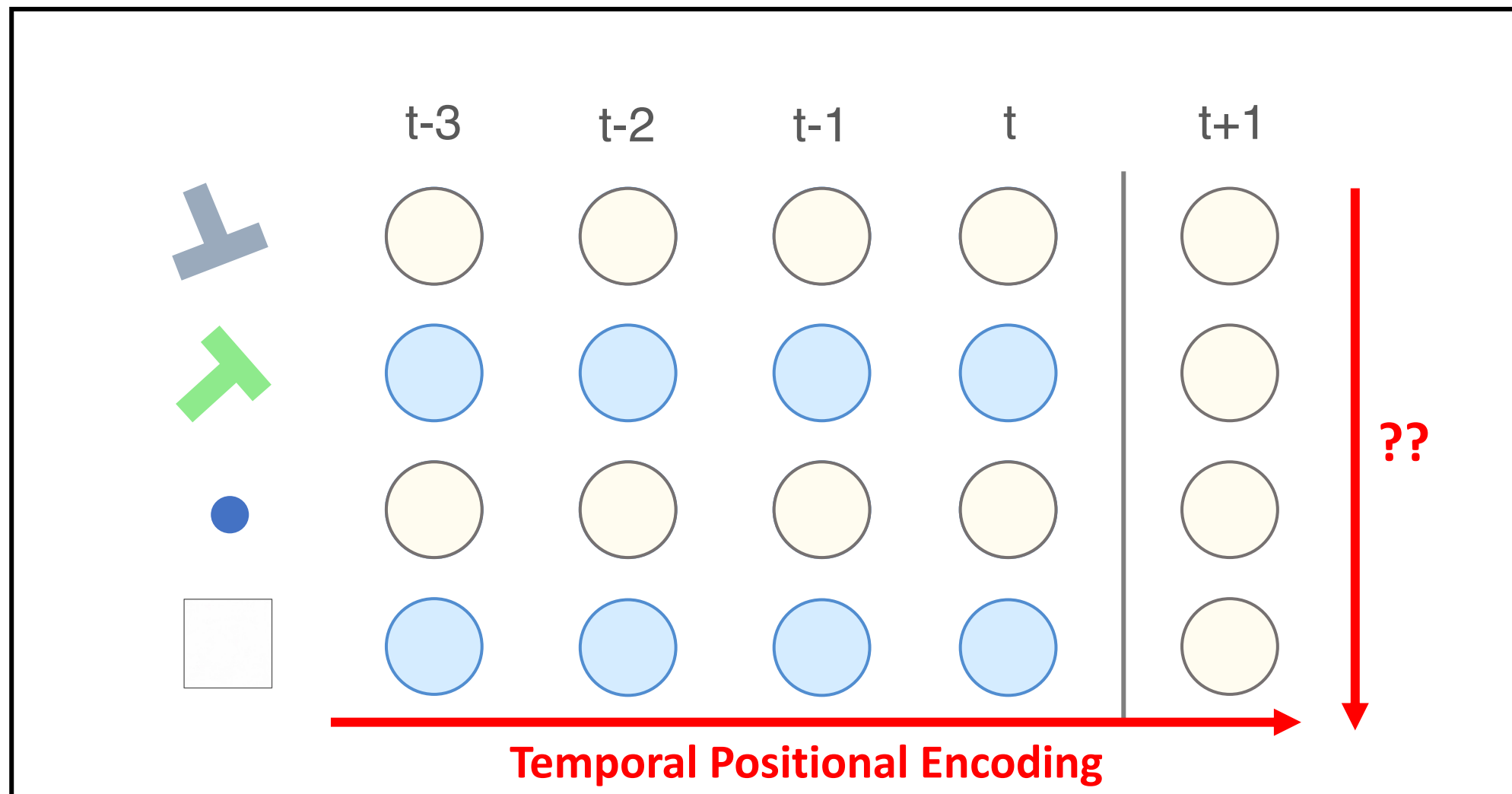


BI-DIRECTIONAL TRANSFORMER

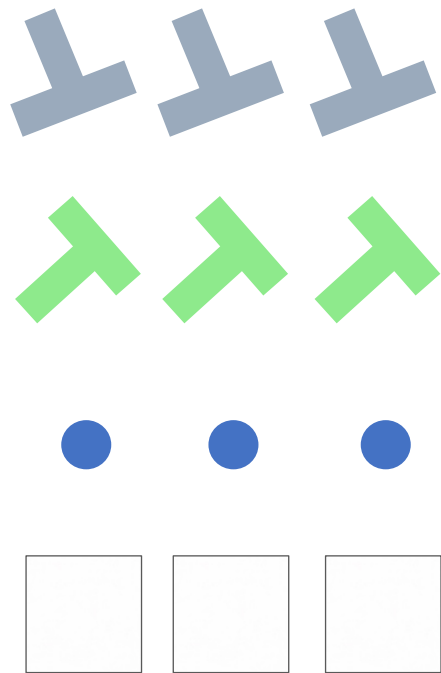




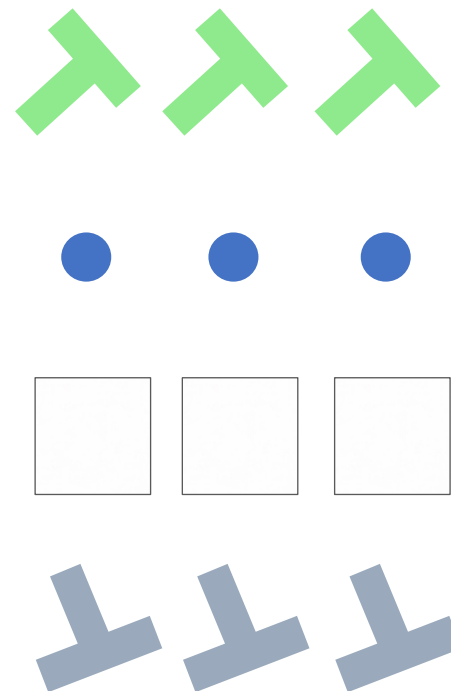




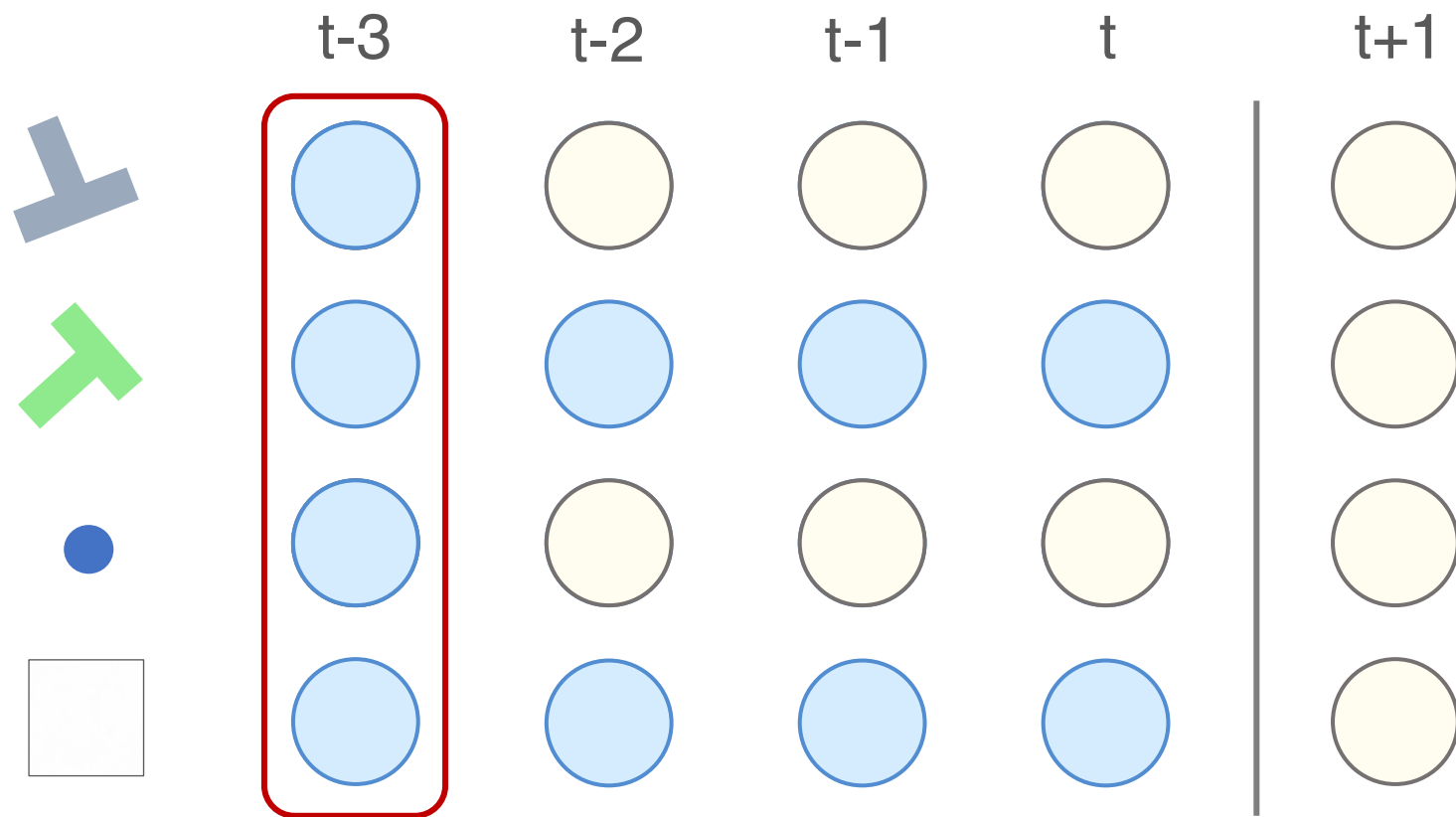
Video A



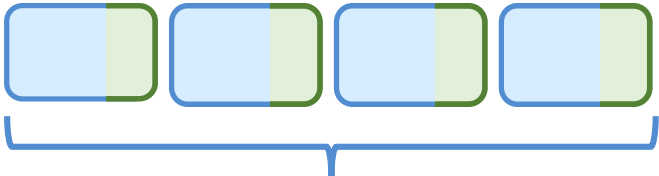
Video B



Mask Token $\bigcirc = \phi(\boxed{\bigcirc}) + \mathbf{e}_\tau$



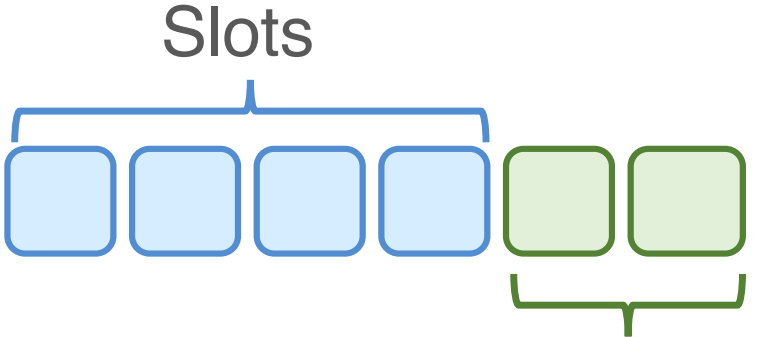
Action conditioning



Slots + Auxiliaries

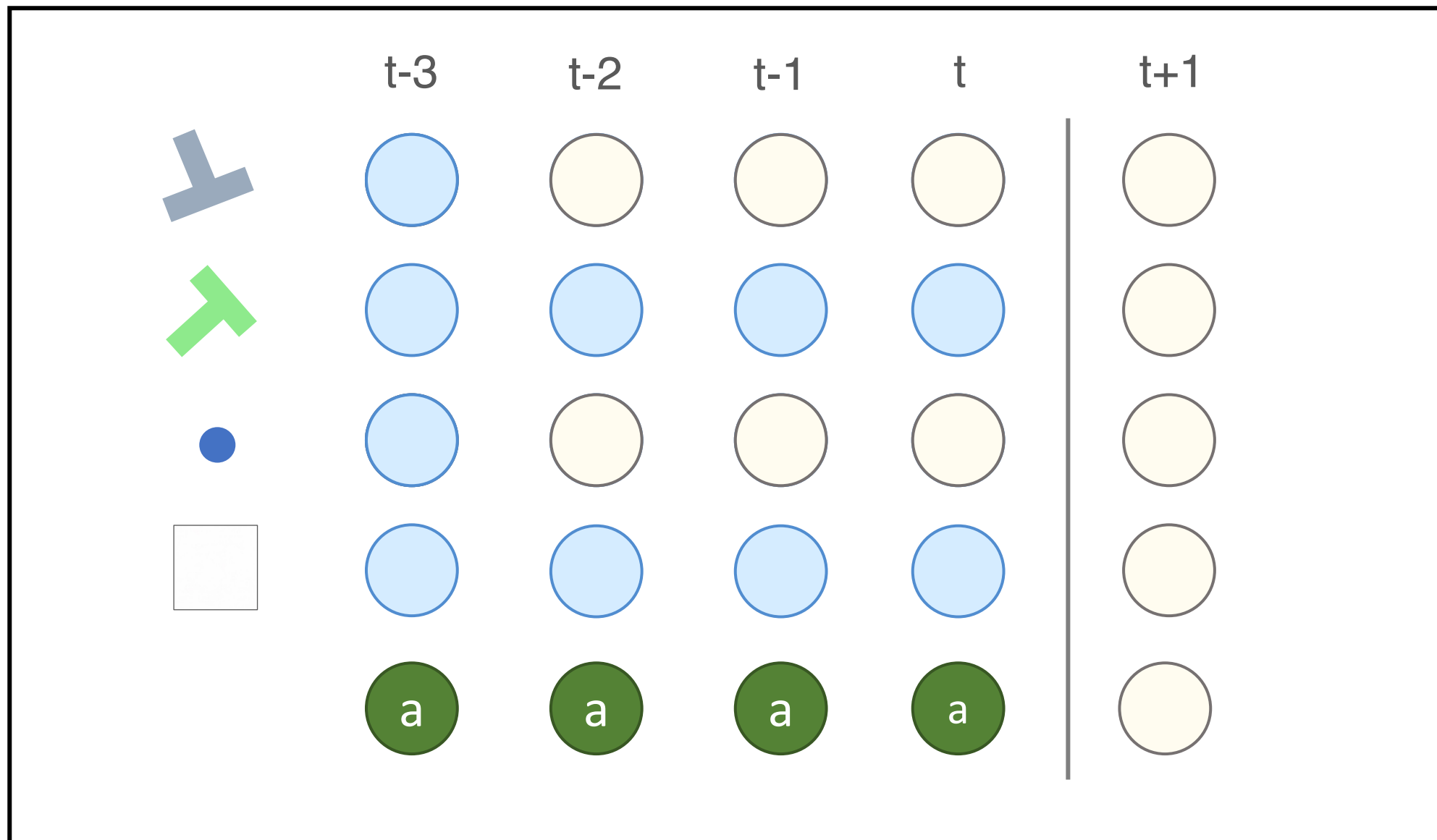
DINO-WM

vs

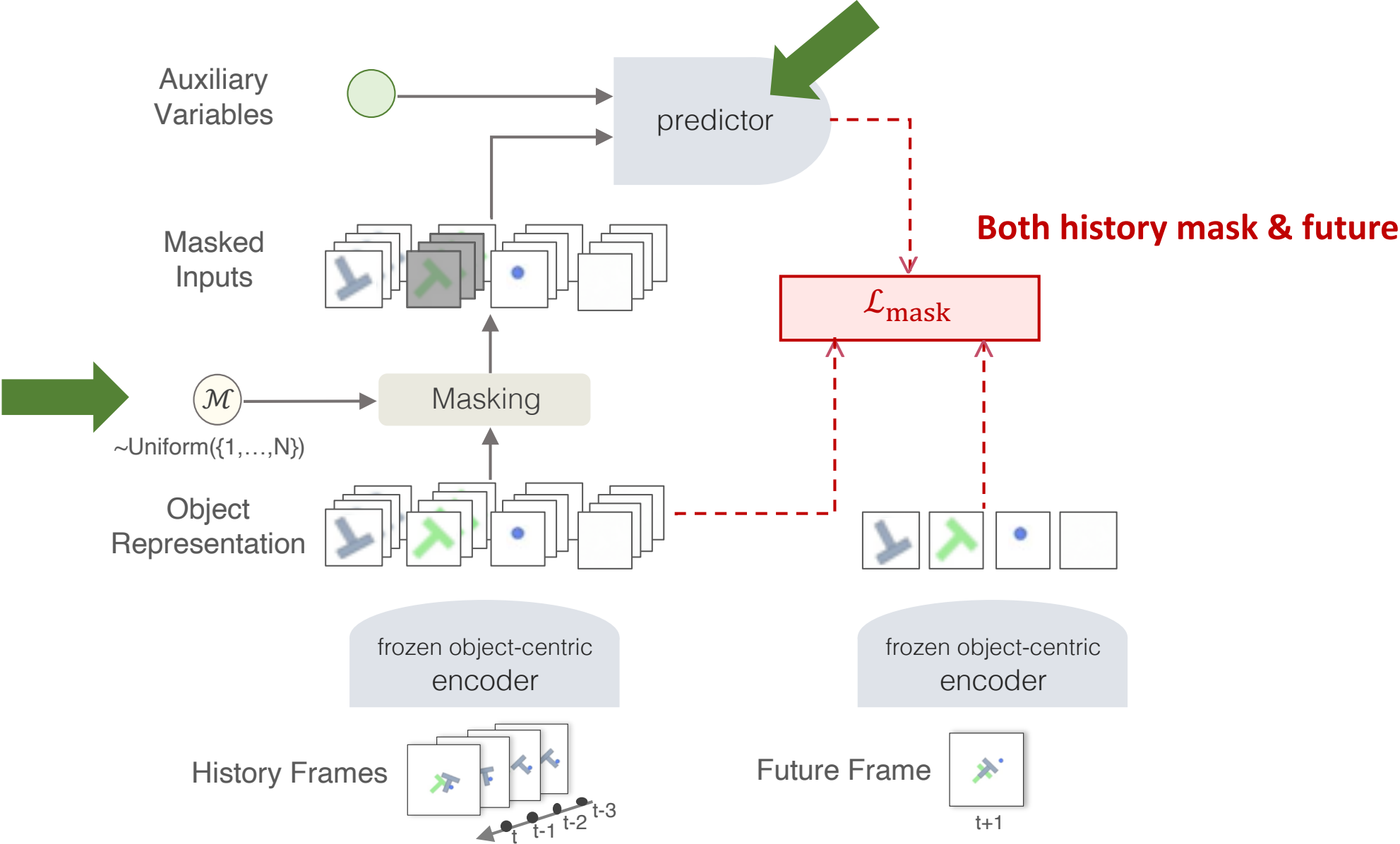


Auxiliaries

C-JEPA



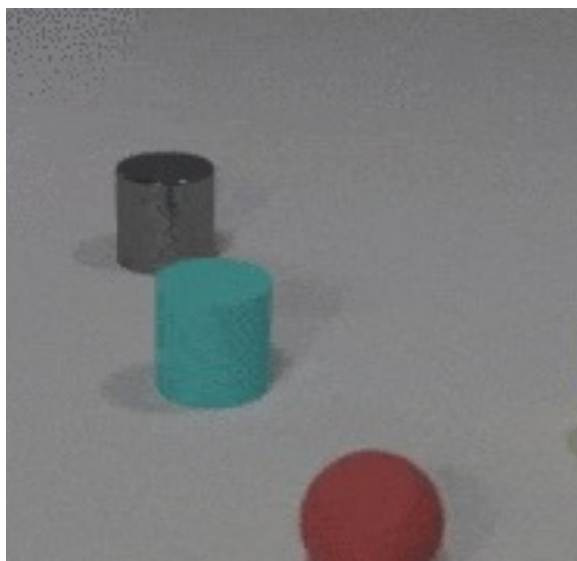
Causal JEPA



- *Reasoning on (counterfactual) questions*
- *Planning and control*
- *Physical plausibility*

CLEVRER – Reasoning

No Mask,
Same Architecture

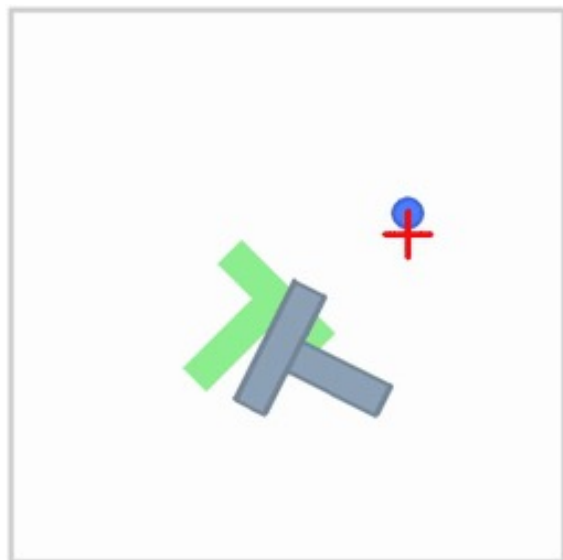


Model	$ \mathcal{M} $	Average per que. (%)	Counterfactual VQA (%)	
			per opt.	per que.
OC-JEPA (V)	0	82.79	79.53	47.68
C-JEPA (V)	1	83.95 $\uparrow$ 1.16	80.34 $\uparrow$ 0.81	49.67 $\uparrow$ 1.99
	2	84.56 $\uparrow$ 1.77	80.61 $\uparrow$ 1.08	50.25 $\uparrow$ 2.57
	3	87.61 $\uparrow$ 4.82	86.49 $\uparrow$ 6.96	63.60 $\uparrow$ 15.92
	4	89.40 $\uparrow$ 6.61	88.67 $\uparrow$ 9.14	68.81 $\uparrow$ 21.13
OC-JEPA (S)	0	77.28	76.69	41.10
C-JEPA (S)	1	78.39 $\uparrow$ 1.11	77.26 $\uparrow$ 0.57	43.13 $\uparrow$ 2.03
	2	83.88 $\uparrow$ 6.60	85.16 $\uparrow$ 8.47	60.19 $\uparrow$ 19.09
	3	79.02 $\uparrow$ 1.74	77.78 $\uparrow$ 1.09	43.77 $\uparrow$ 2.67
	4	73.28 $\downarrow$ 4.00	73.55 $\downarrow$ 3.14	34.06 $\downarrow$ 7.04

V: VideoSAUR

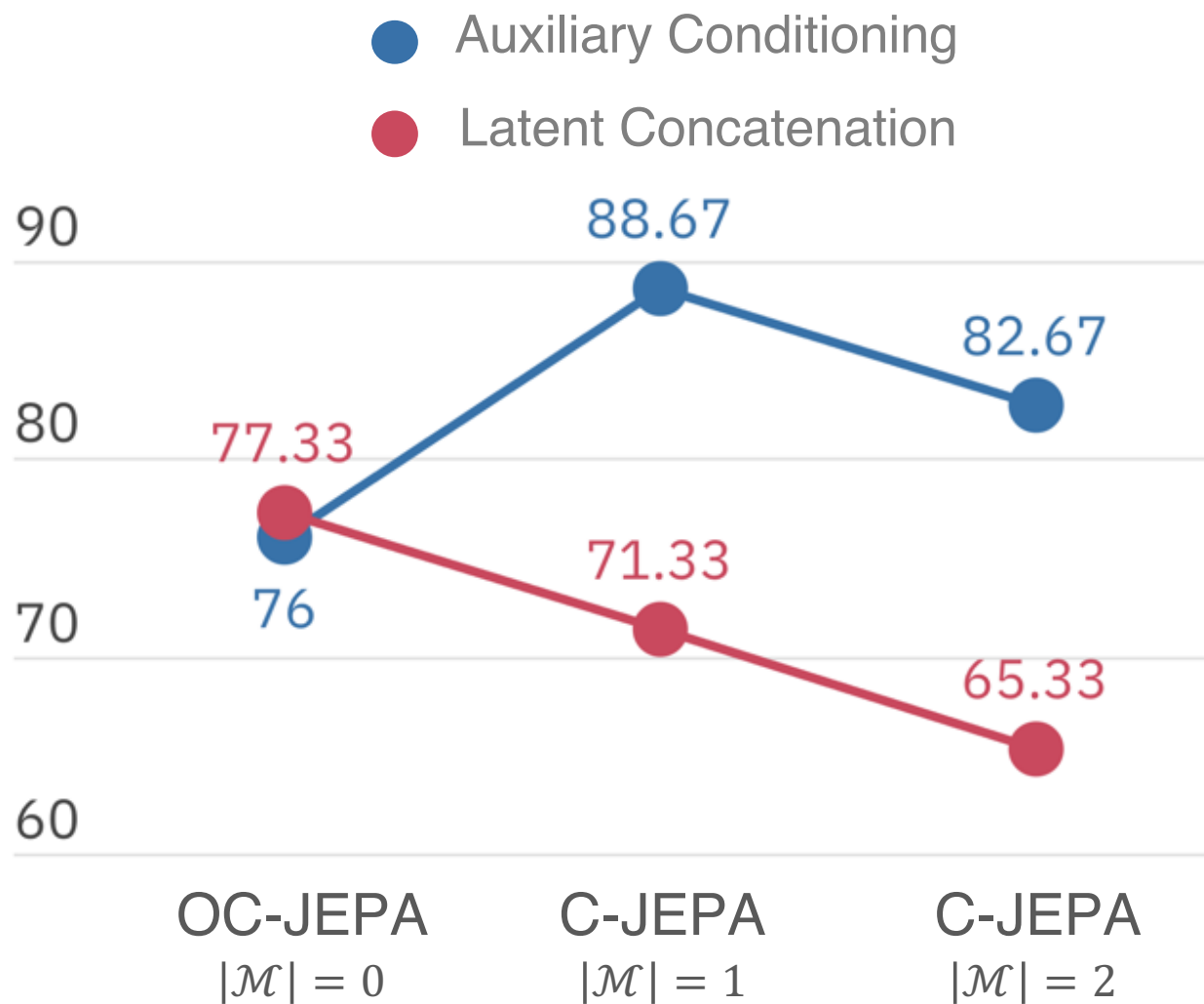
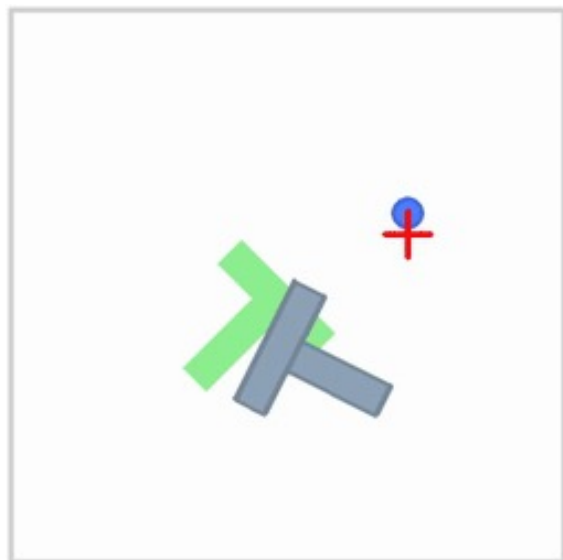
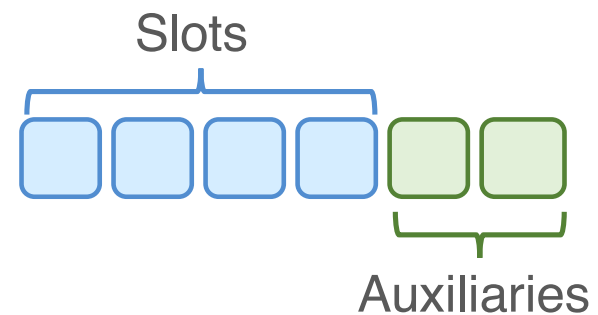
S: SAVi

PushT – Planning



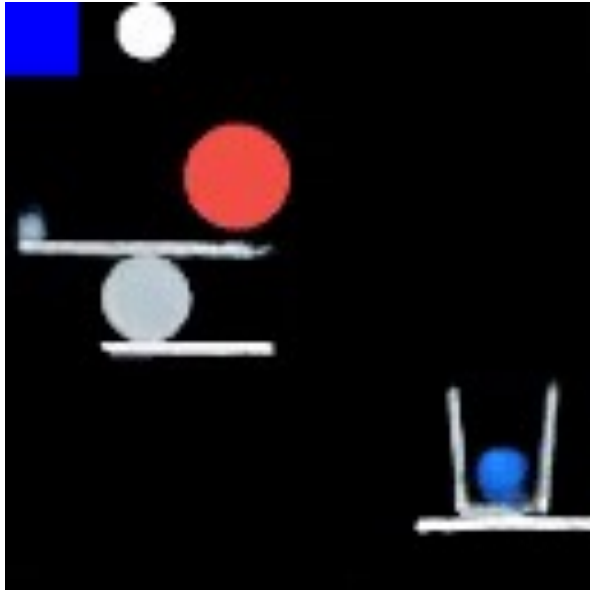
# Token $\times d$	Model	Success Rate (%)
196×384	DINO-WM	91.33
	DINO-WM-Reg.	88.00
6×128	OC-DINO-WM	60.67
	OC-JEPA	76.00 $\uparrow +15.33$
	C-JEPA	<u>88.67</u> $\uparrow +28.00$

PushT - Planning



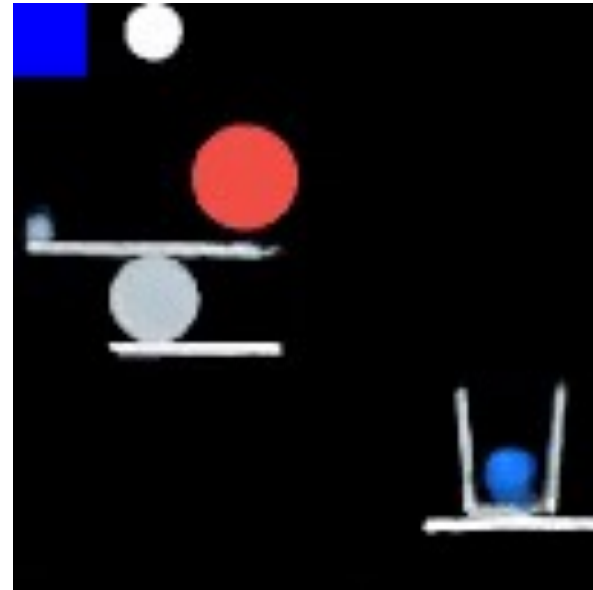
PHYRE - Rollout

Learning Correlation vs.



OC - JEPA

Learning Dynamics



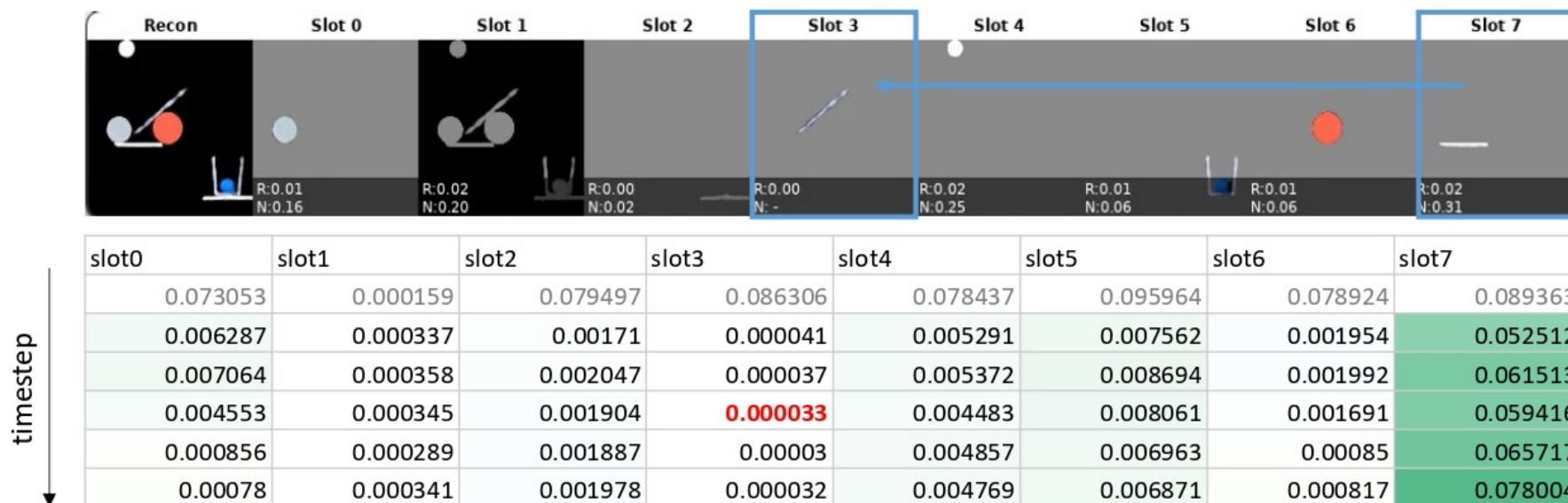
C - JEPA

PHYRE – Rollout

OC-JEPA



C-JEPA



causal

=temporally directed predictive dependencies

The role of object masking:
Predictor finds “Influence Neighborhood”
(= predictively sufficient sets)

What we assume

1. No instantaneous causality
2. Shared mechanism
3. Object Aligned Representation
4. History sufficiency

What we do not assume

1. First order markov process
2. No confounder

1. Good State Representation

what matters now

Object
Representation

2. Good Transition Model

how the world evolves

Model Learns
Predictive Sufficiency

3. Good Dynamics Model

how the world responds to you

Action as
Auxiliary Variable

A simple world model

Lucas Maes

Invited Talk @ NVIDIA

April 6th 2026

work done with Quentin Le Lidec*, Damien Scieur, Yann LeCun, Randall Balestriero

What is a world model?

A world model is mapping modeling the state evolution of a system (dynamics) under some control variables.

$$f: \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$$

The term World Models has been initially introduced by Ha & Schmidhuber, 2018. While the concept has been there since the 50s.

World Models

David Ha¹ Jürgen Schmidhuber^{2,3}

Abstract

We explore building generative neural network models of popular reinforcement learning environments. Our *world model* can be trained quickly in an unsupervised manner to learn a compressed spatial and temporal representation of the environment. By using features extracted from the world model as inputs to an agent, we can train a very compact and simple policy that can solve the required task. We can even train our agent entirely inside of its own hallucinated dream generated by its world model, and transfer this policy back into the actual environment.

An interactive version of this paper is available at <https://worldmodels.github.io>

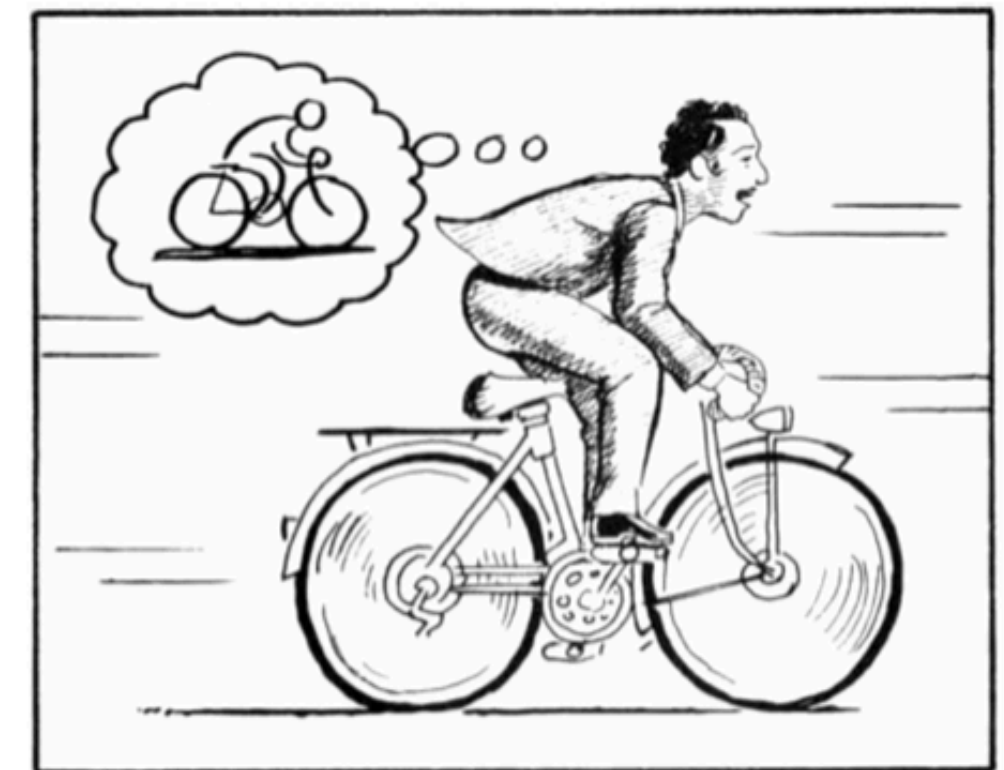


Figure 1. A World Model, from Scott McCloud's *Understanding Comics*. (McCloud, 1993; E, 2012)

[Craig 1943]

[Richalet et al. 1978]

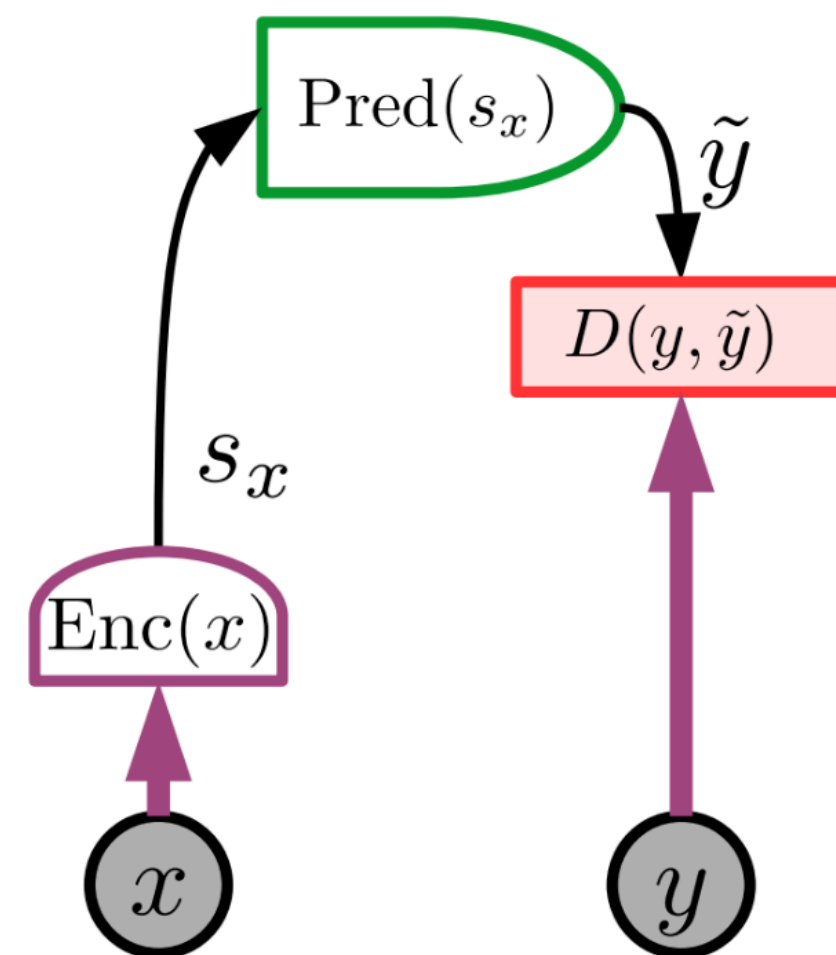
[Ha & Schmidhuber 2018]

JEPA

Joint Embedding Predictive Architecture

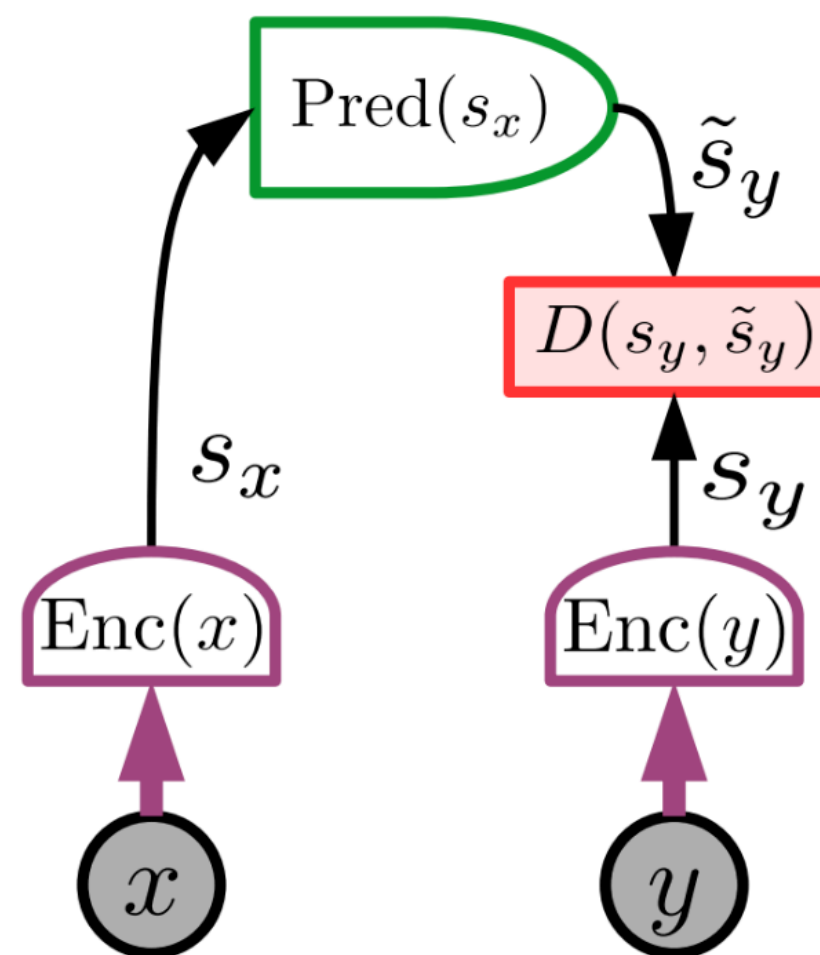
Generative Modelling is not desirable for most of tasks! It forces you to predict every details of your input distributions even when not necessary

JEPA makes prediction in **an abstract representation space** getting rid of irrelevant details



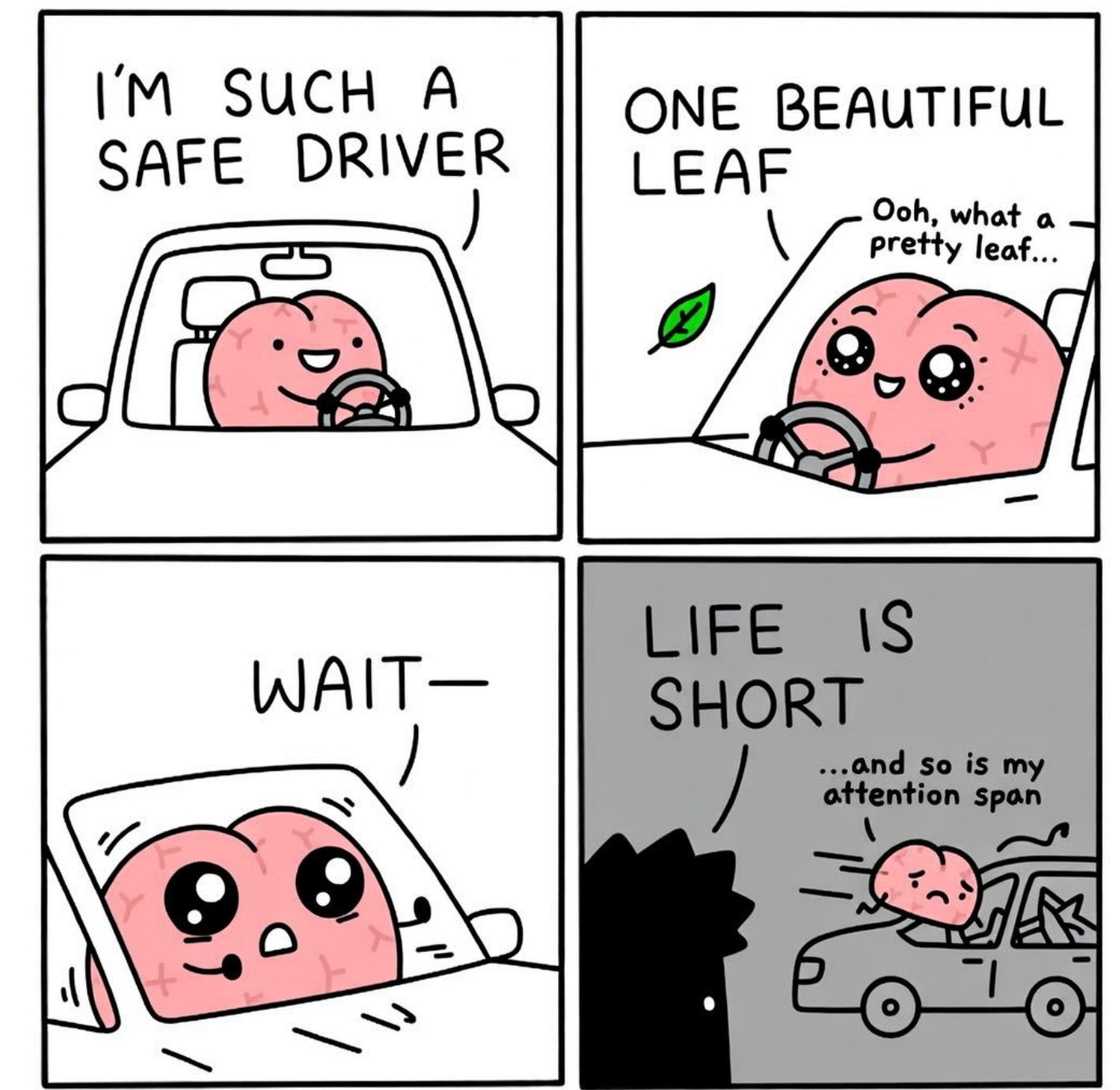
Generative Architecture

(VAE, MAE, ...)



Joint Embedding

Predictive Architecture

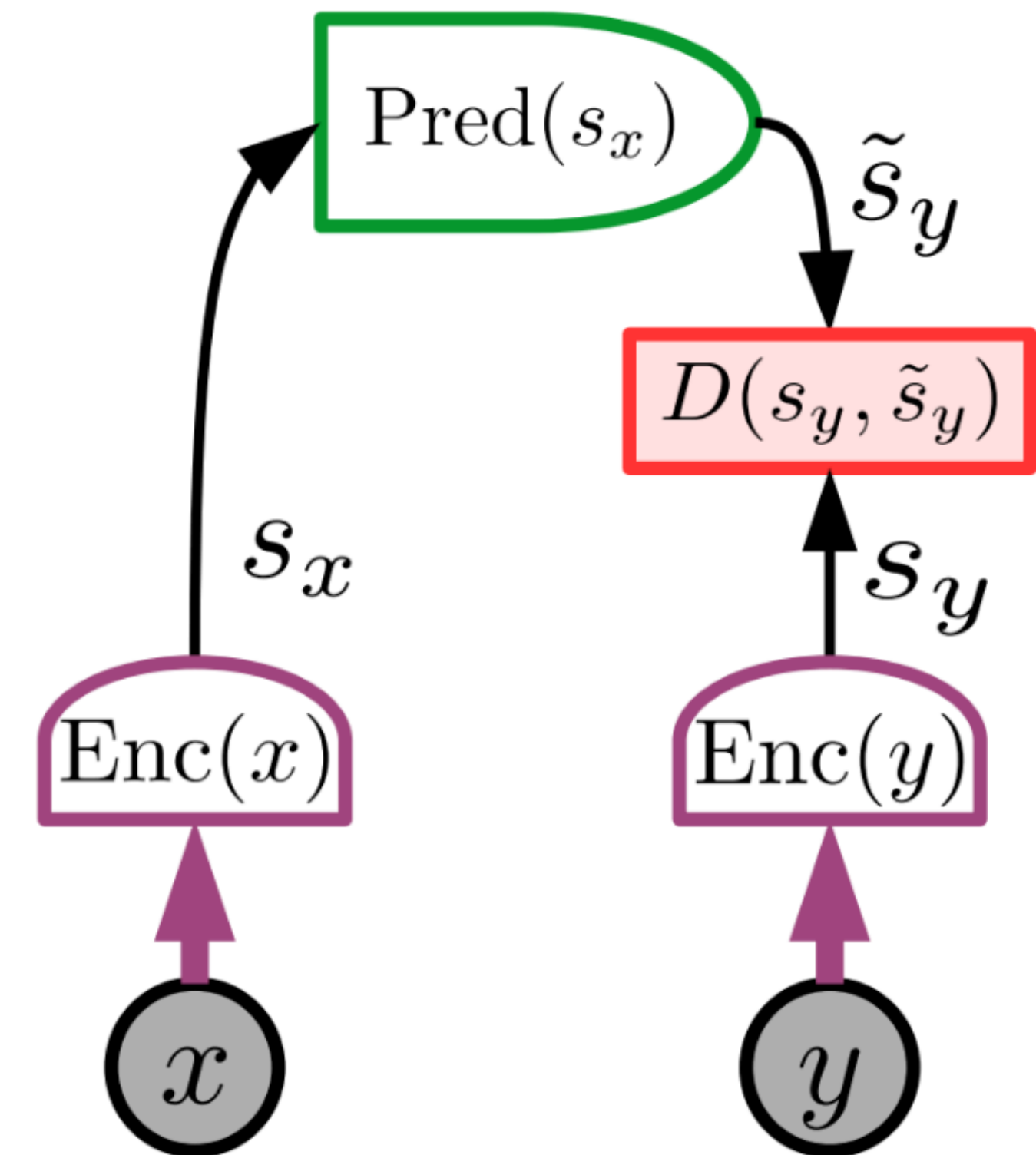
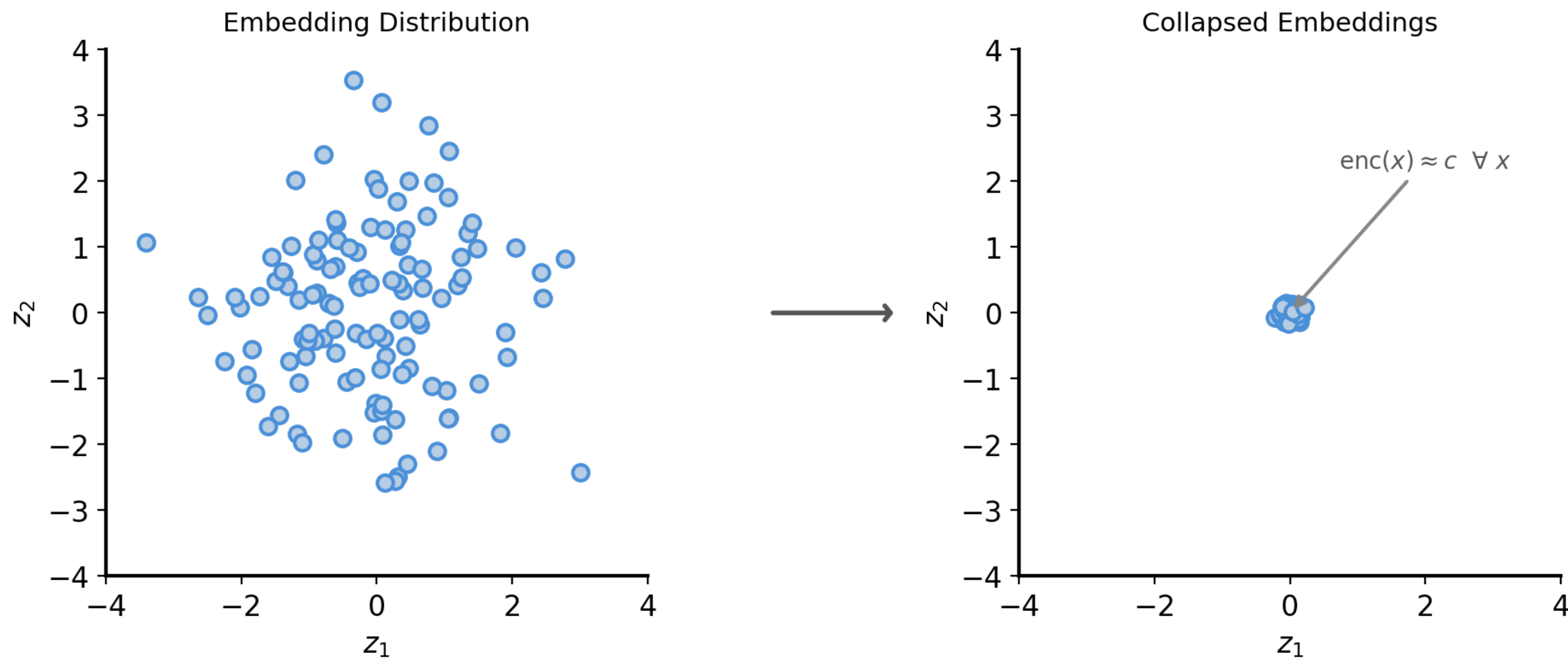


[LeCun 2022]

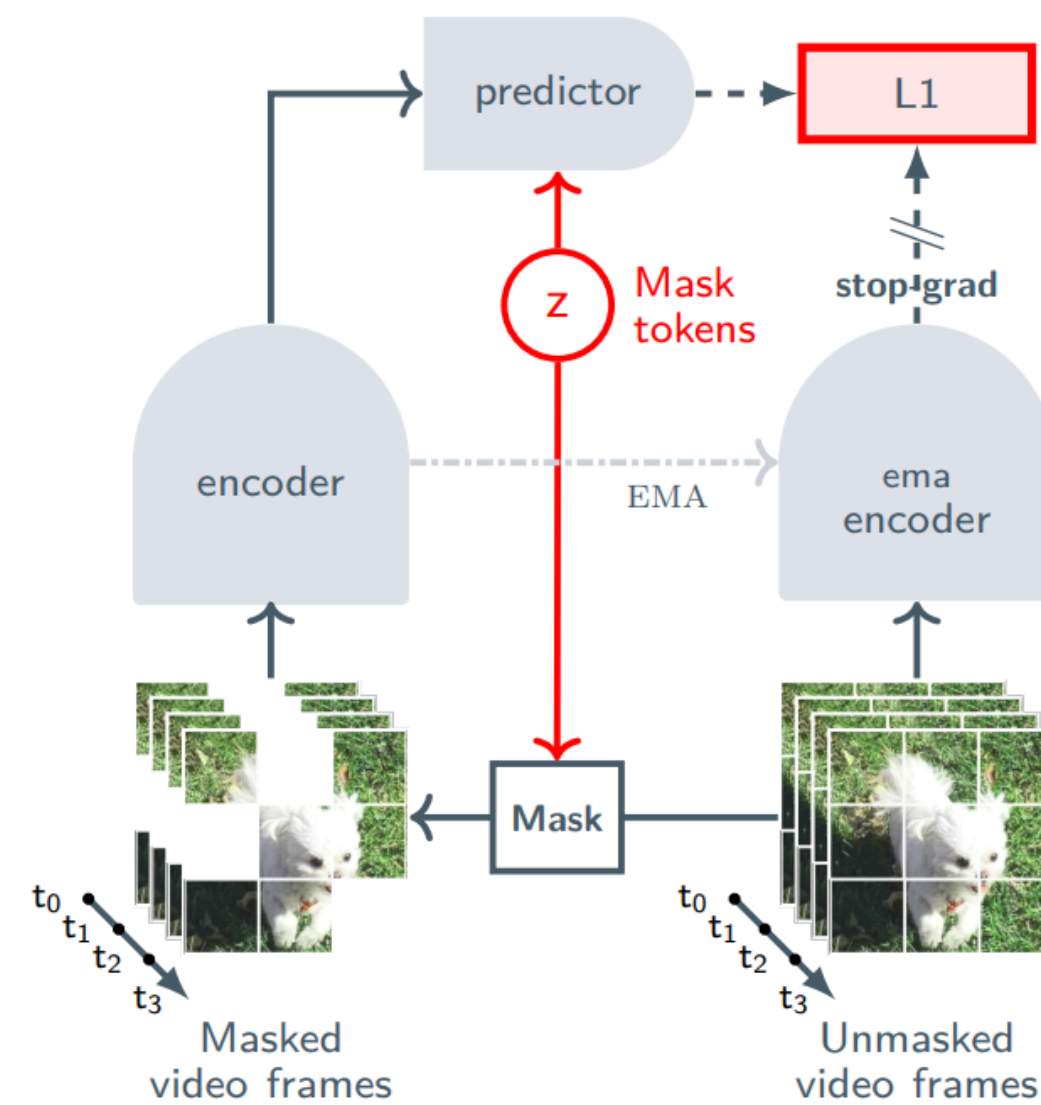
Collapse

Just predicting in latent space suffer from collapse.

The encoder trivially learn to predict a constant representation, regardless of the input

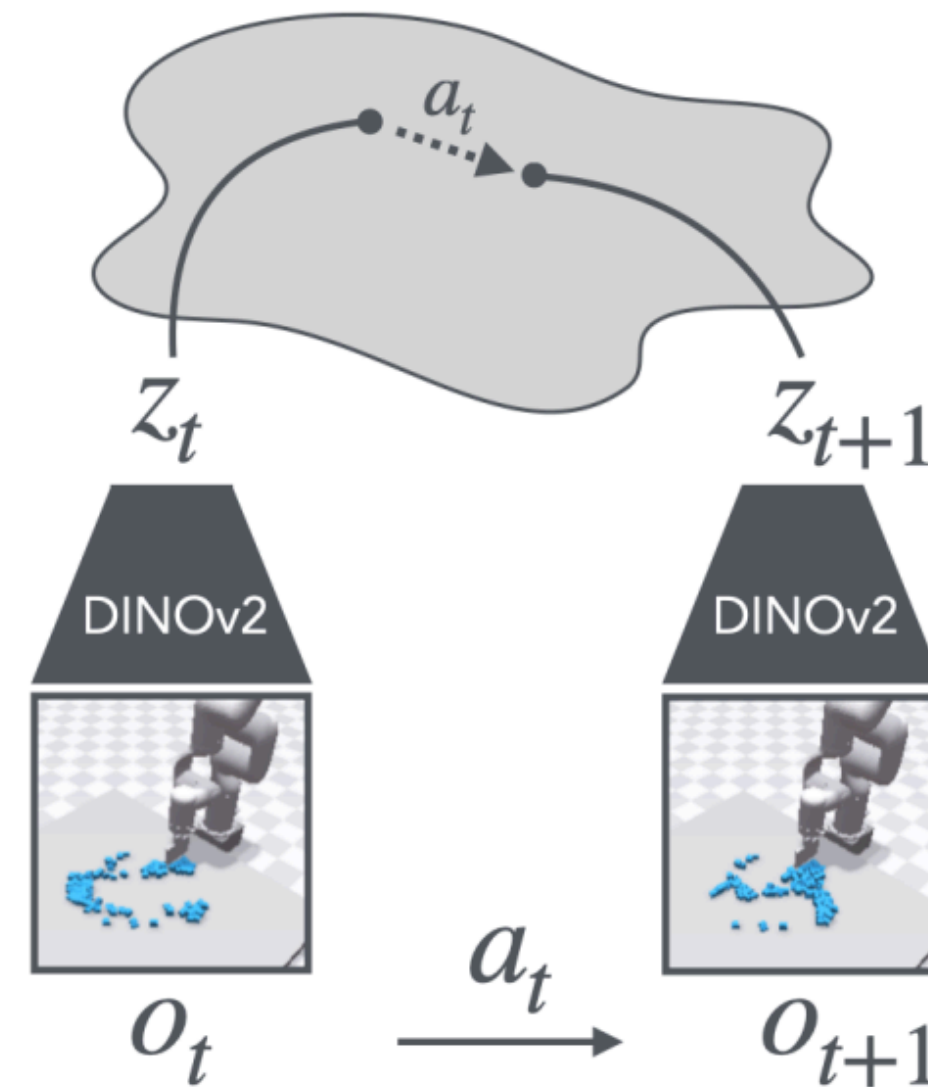


Previous JEPA recipes



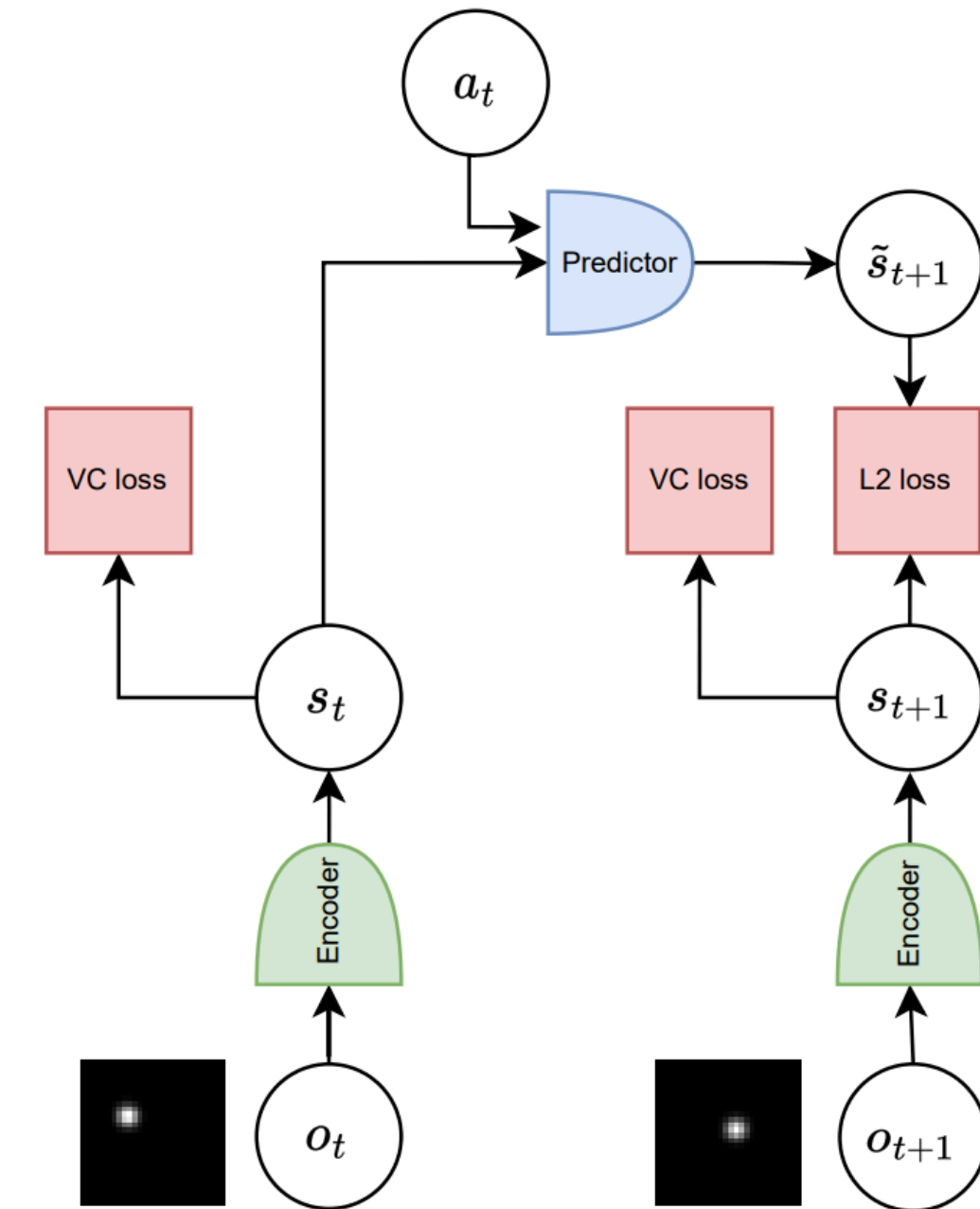
[V-JEPA]

Use making and EMA



[DINO-WM]

Pre-trained encoder
and proprioception



[PLDM]

Use 6 terms loss
(temporal & spatial VCReg)

LeWorldModel:

A Stable End-to-End Joint Embedding Predictive Architecture from Pixels

No EMA

No Masking

No stop-gradient

No pre-trained encoder

No unstable loss

Full End-to-End

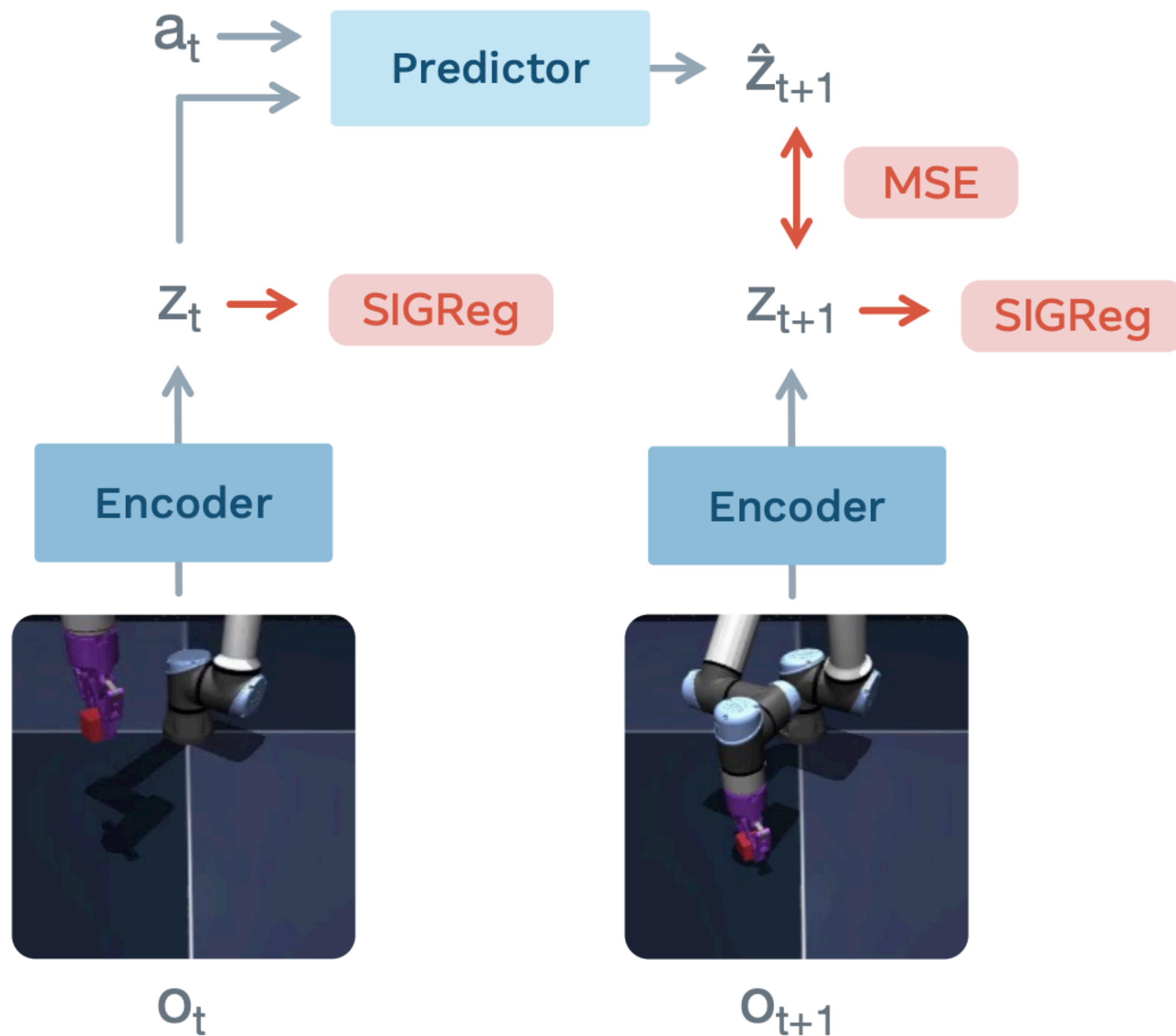
From Raw Pixels

Single-GPU

~50x Faster Planning

1 hyper-parameter

LeWorldModel

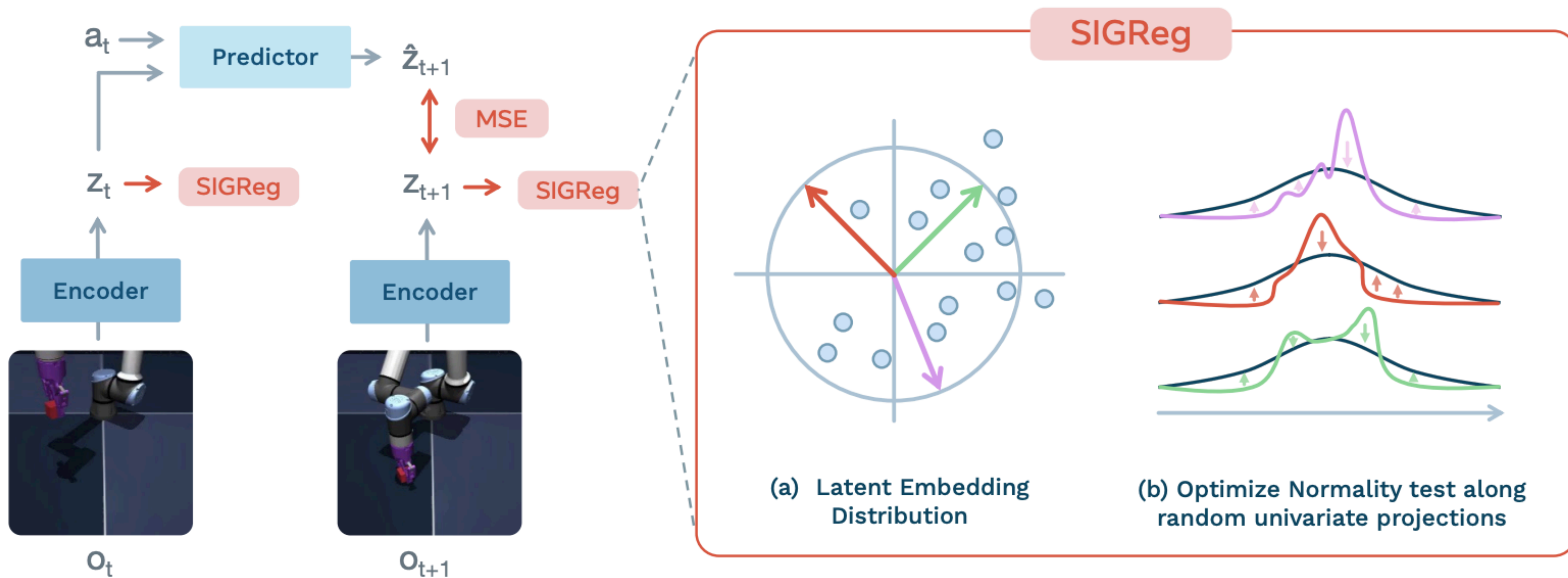


what you see is what you get!

```
def LeWorldModel(obs, actions, lambd=0.1):  
    """  
    obs: (B, T, C, H, W) raw pixels sequence  
    actions: (B, T, A) action sequence  
    lambd: (float) SIGReg loss weight  
    """  
  
    emb = encoder(obs) # (B, T, D)  
    next_emb = predictor(emb, actions) #(B, T, D)  
  
    # — LeWorldModel training loss  
  
    # next-embedding prediction loss  
    pred_loss = F.mse_loss(emb[:, 1:] - next_emb[:, :-1])  
  
    # step-wise sigreg (anti-collapse)  
    sigreg_loss = mean(SIGReg(emb.transpose(0, 1)))  
  
    return pred_loss + lambd * sigreg_loss
```


LeWorldModel

Sketched Isotropic Gaussian Regularizer

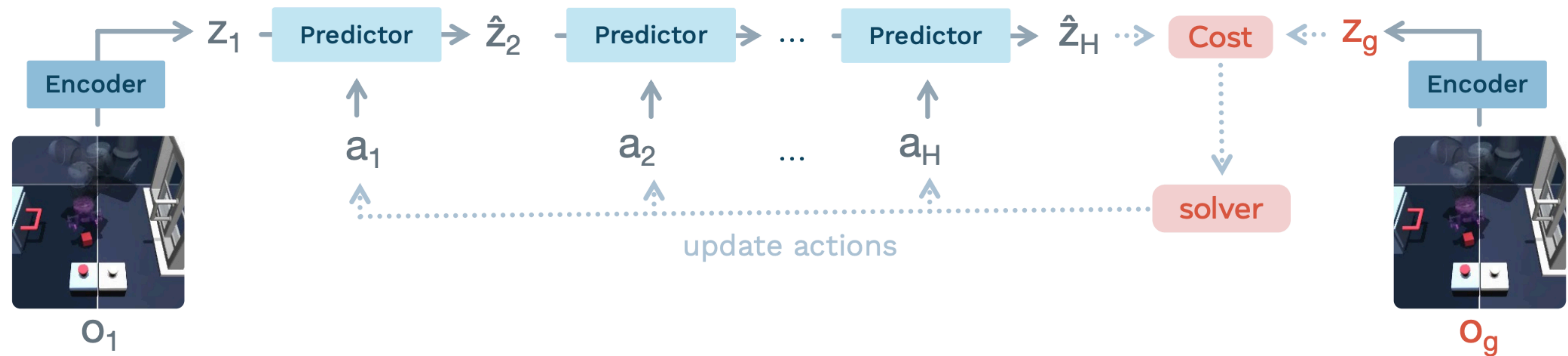


Online Control

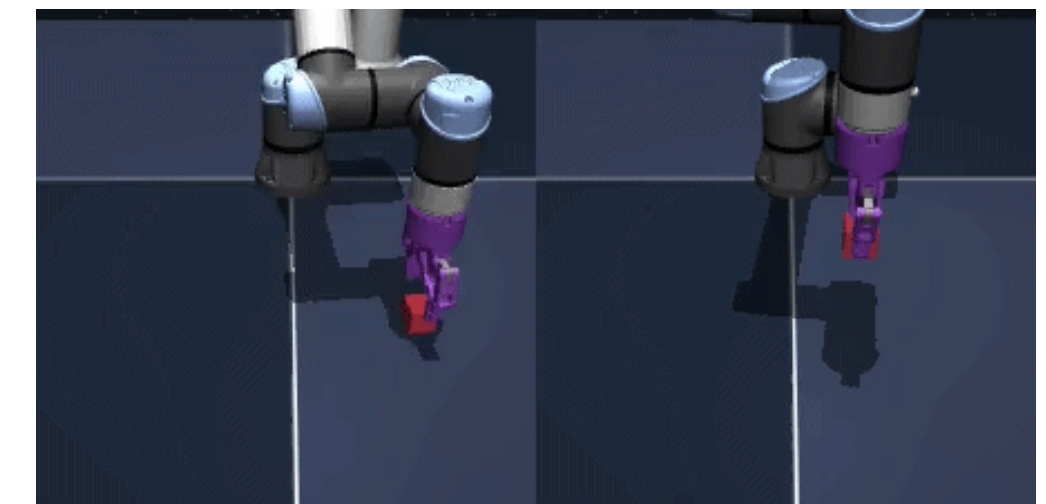
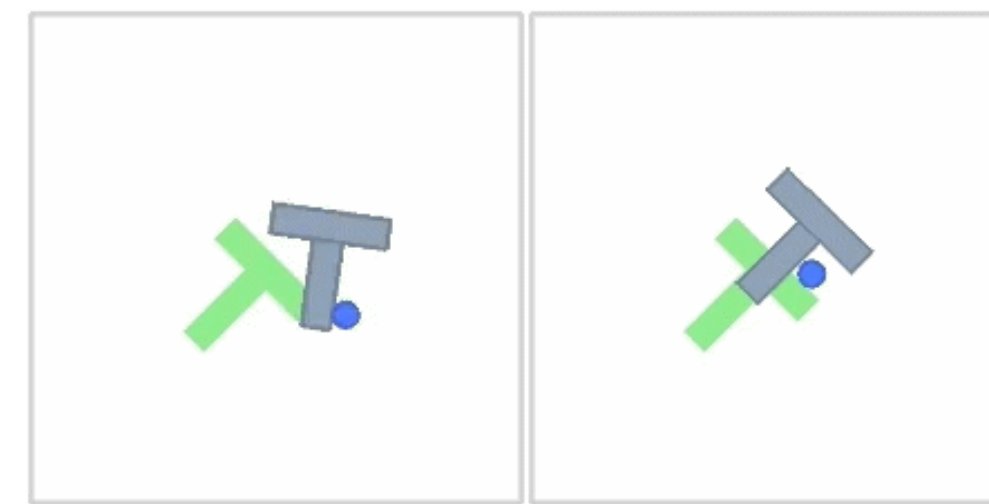
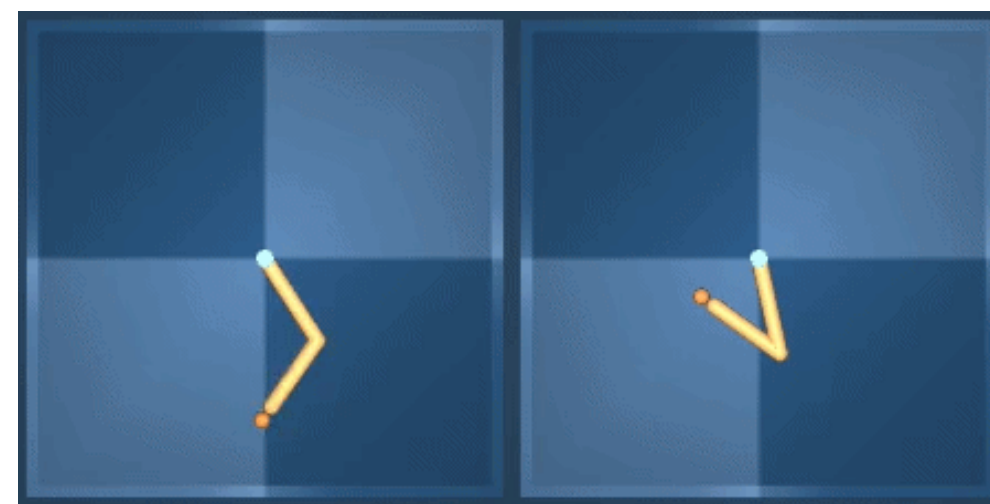
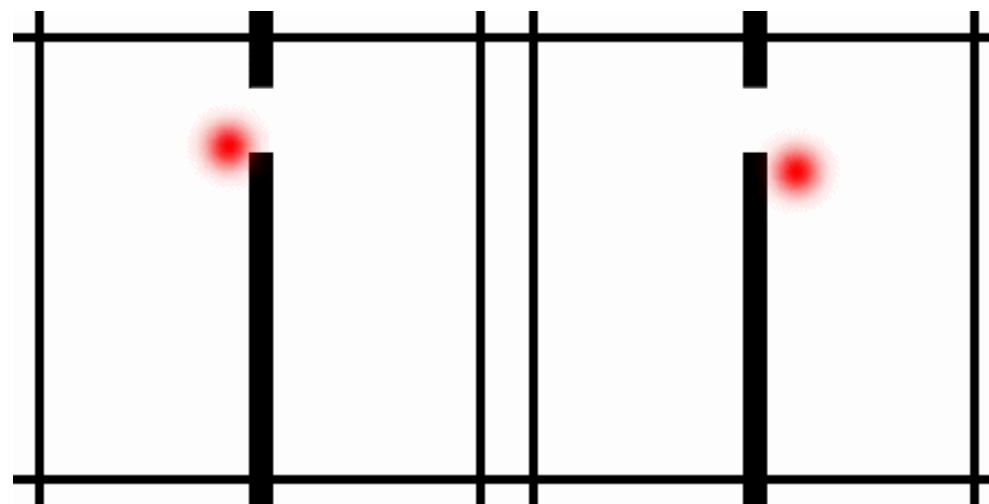
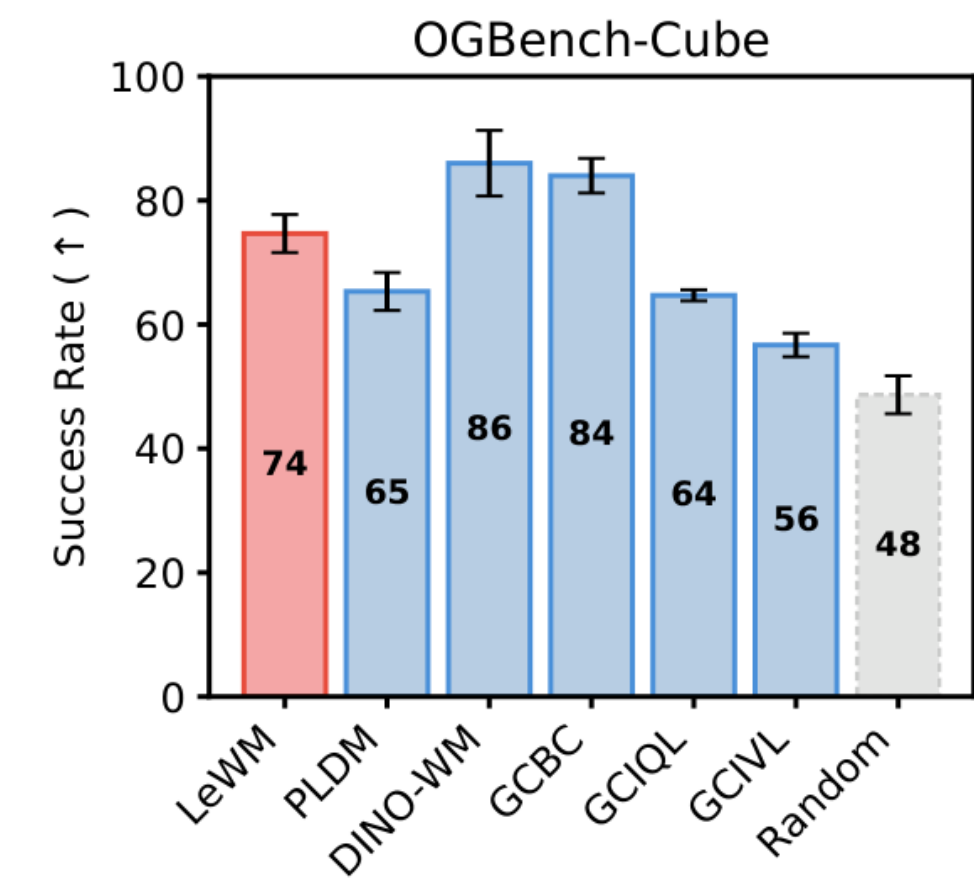
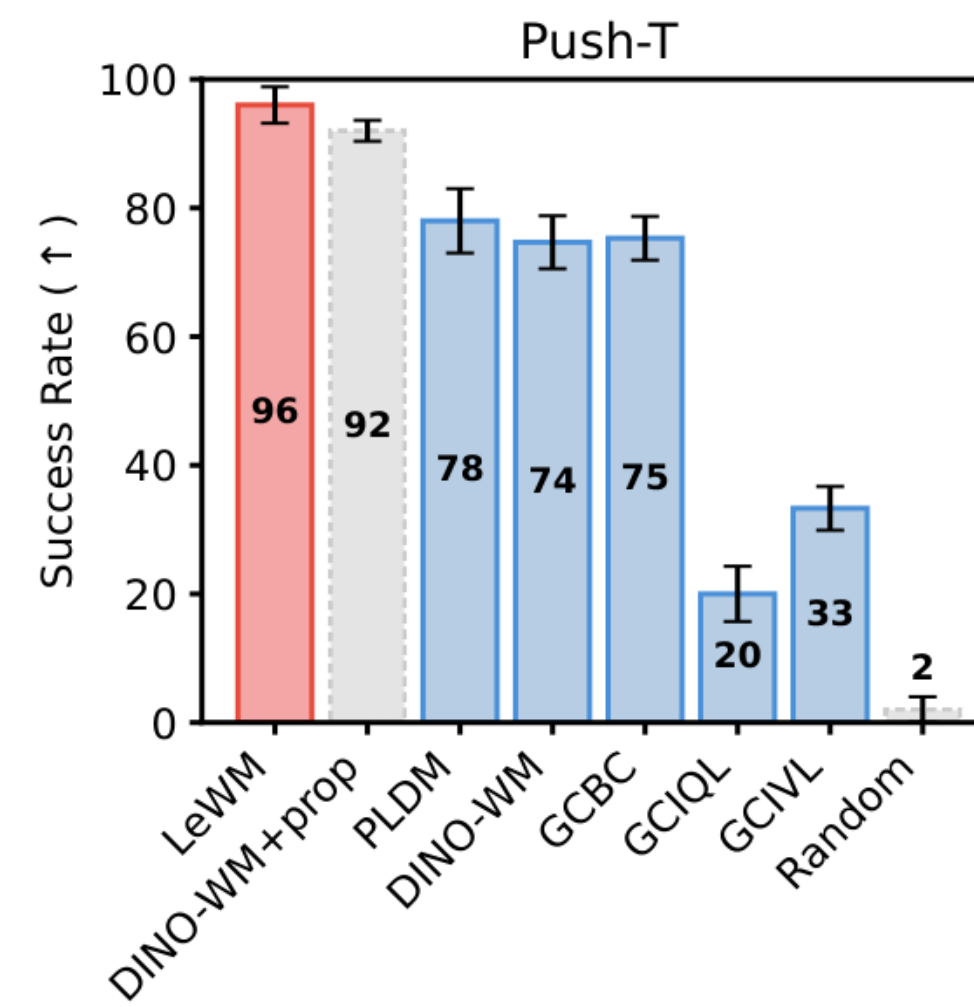
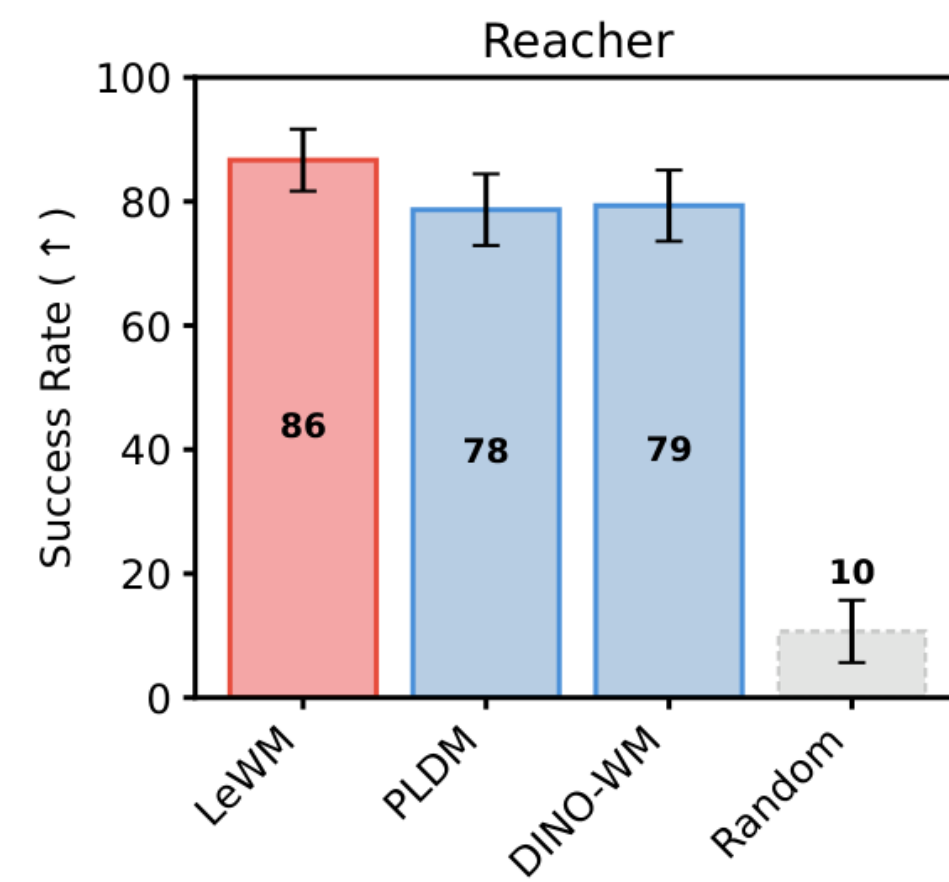
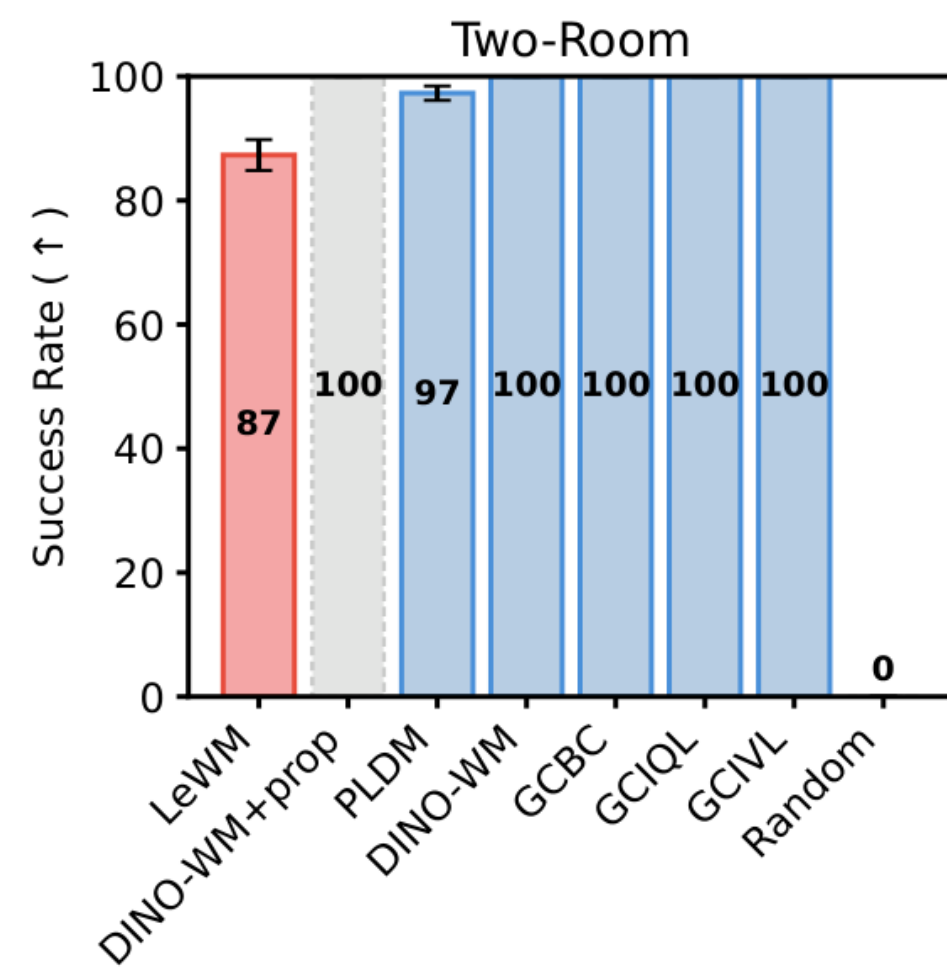
Control

Can you leverage the learned world model to perform online control?

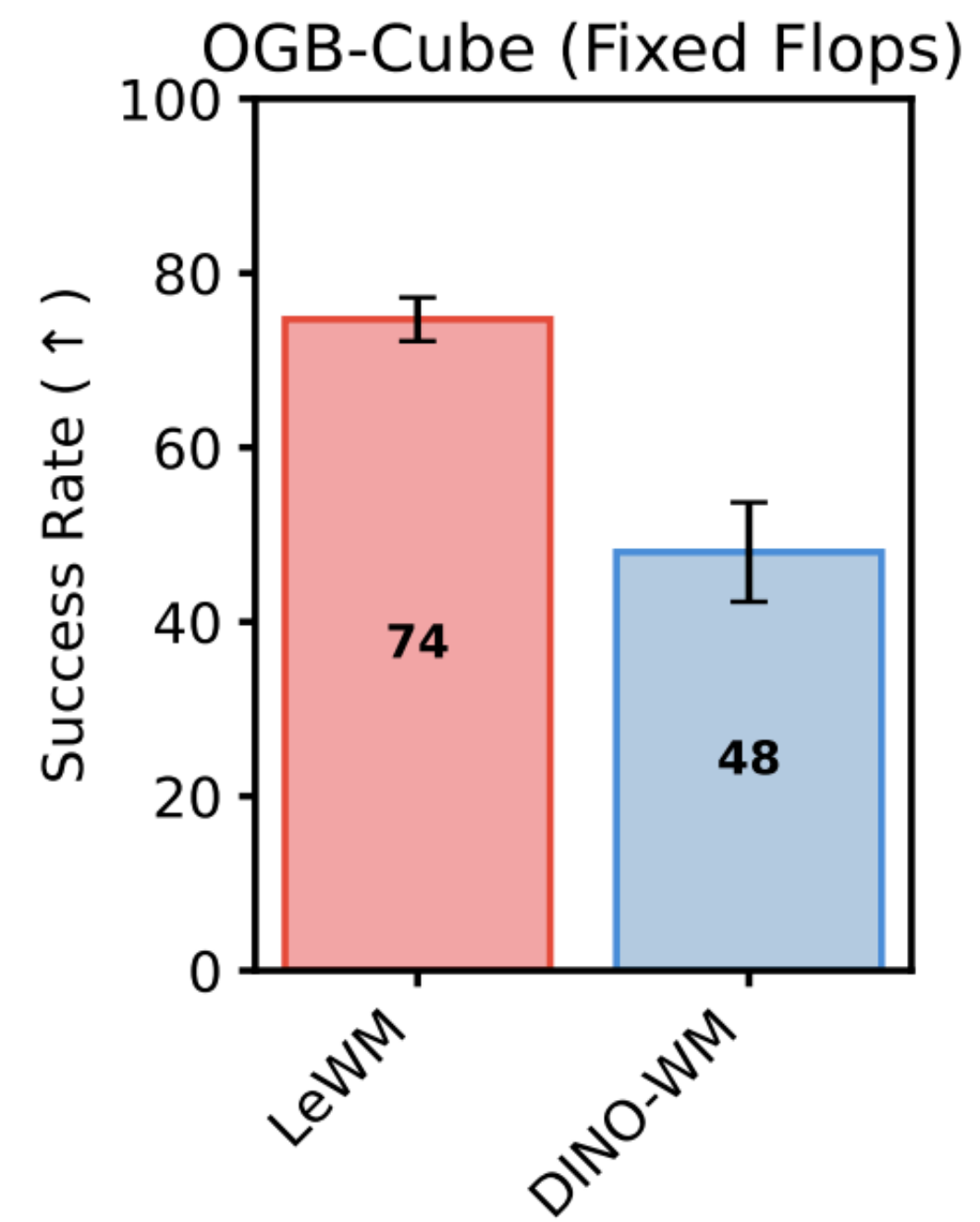
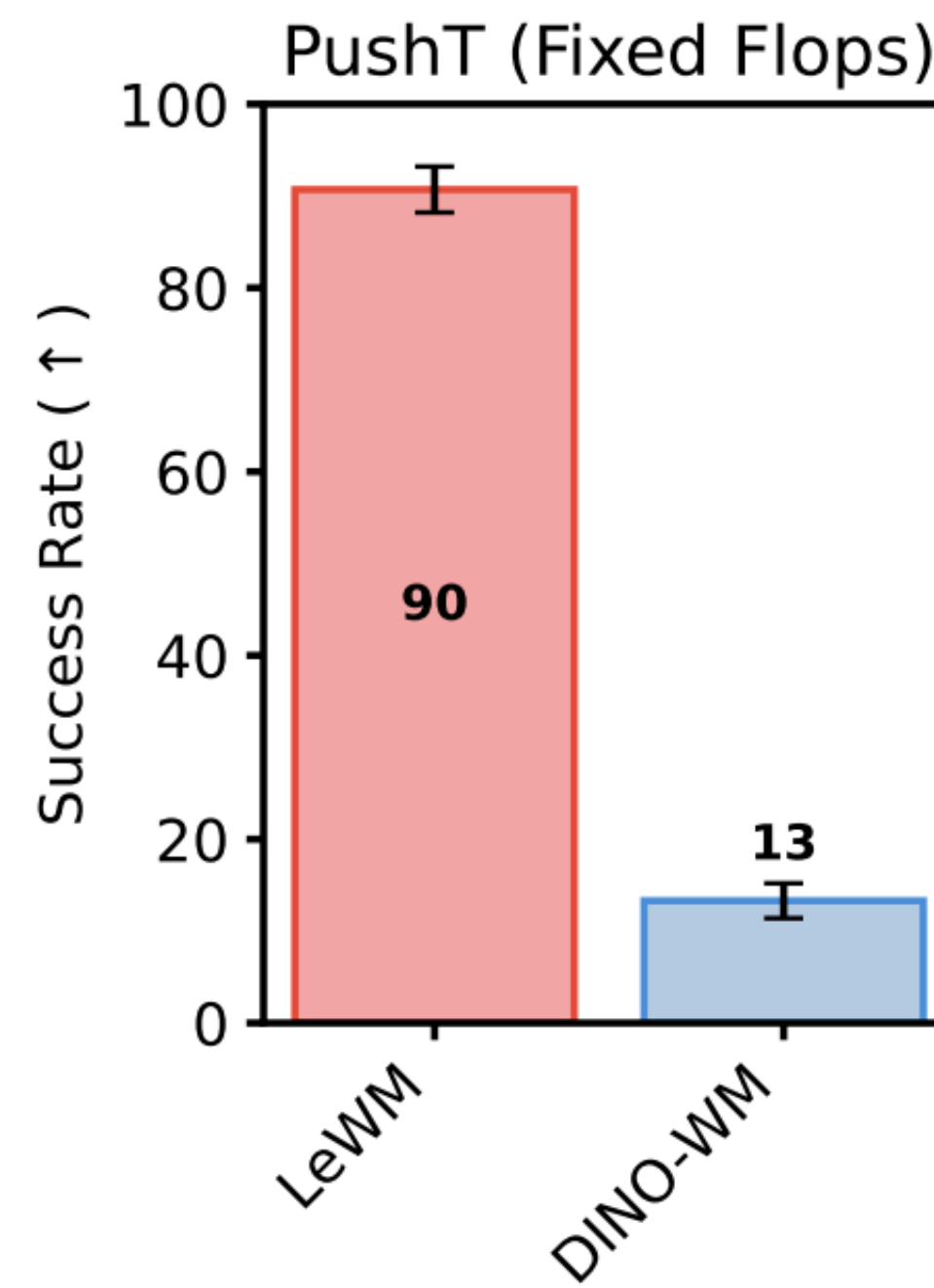
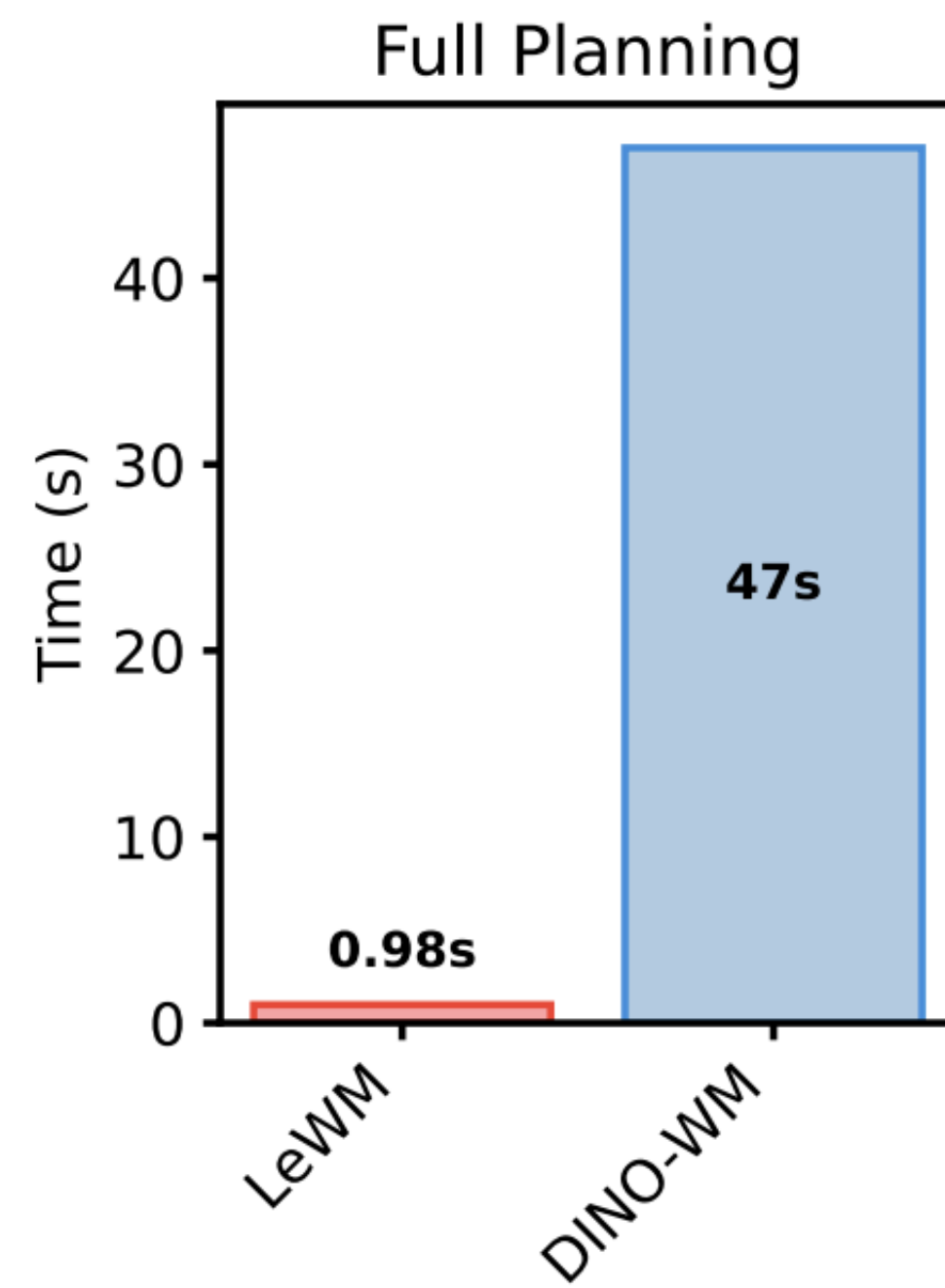
We consider the planning same setup as DINO-WM, PLDM, V-JEPA2 AC, ...



Control



Control

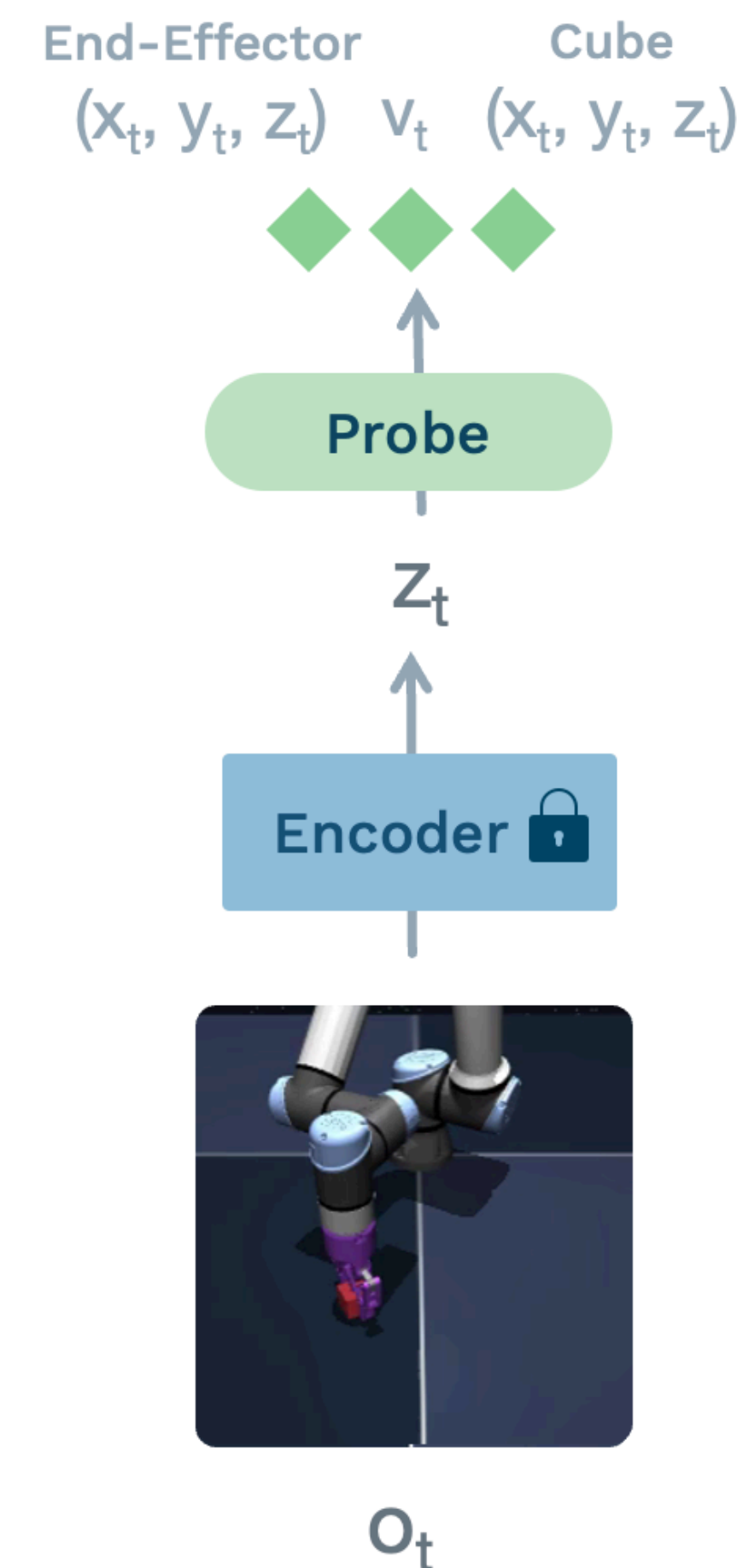
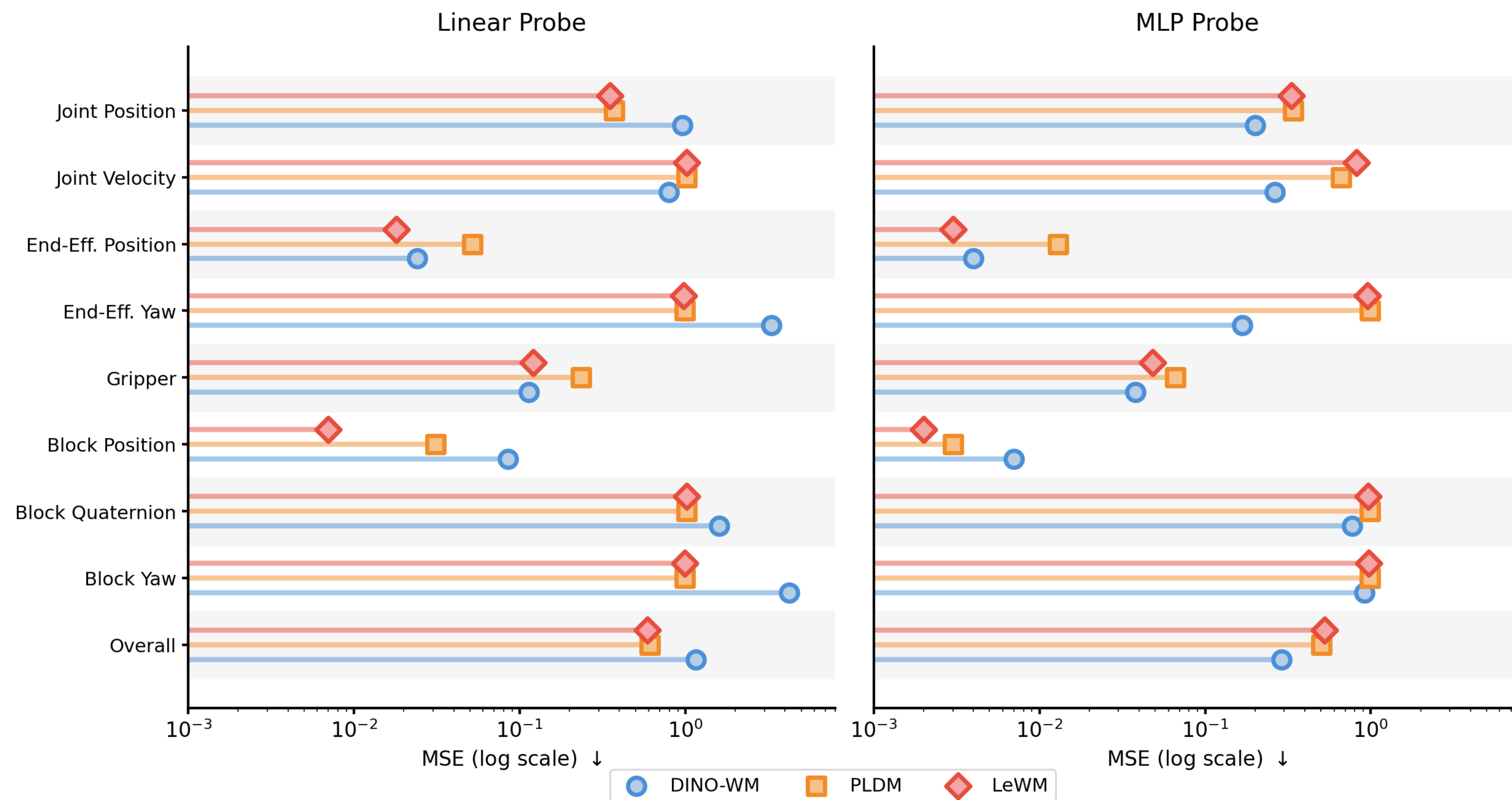


Intuitive Physics Understanding

Intuitive Physics Understanding

Latent probing

OGBench - Cube



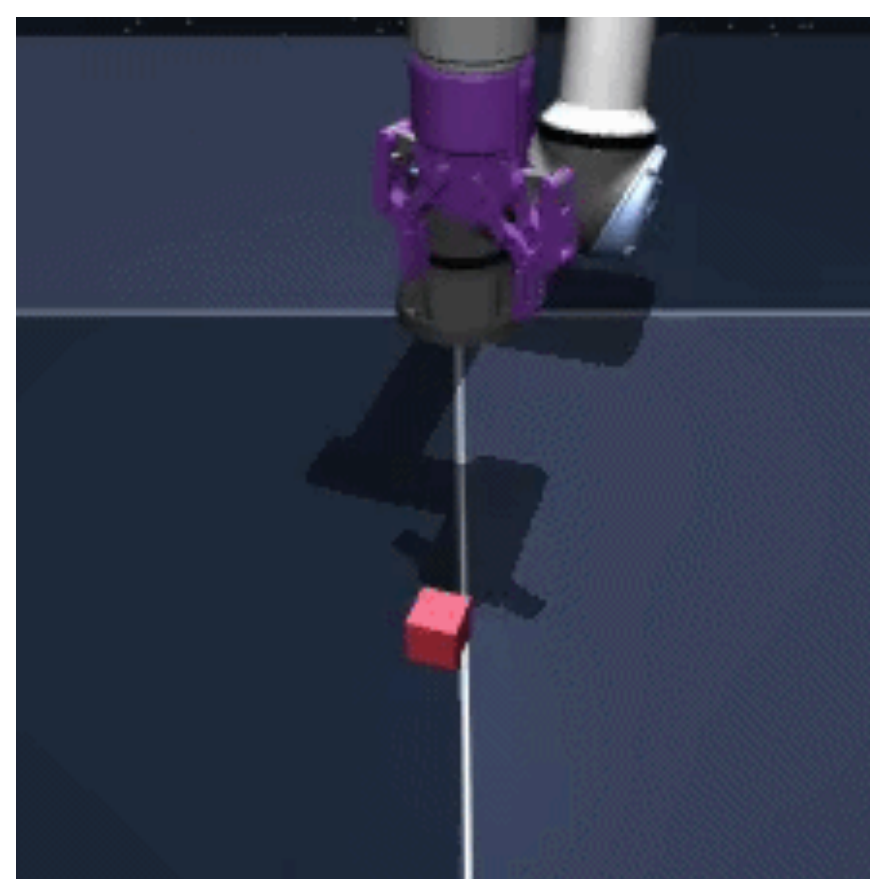
Intuitive Physics Understanding

World model violation

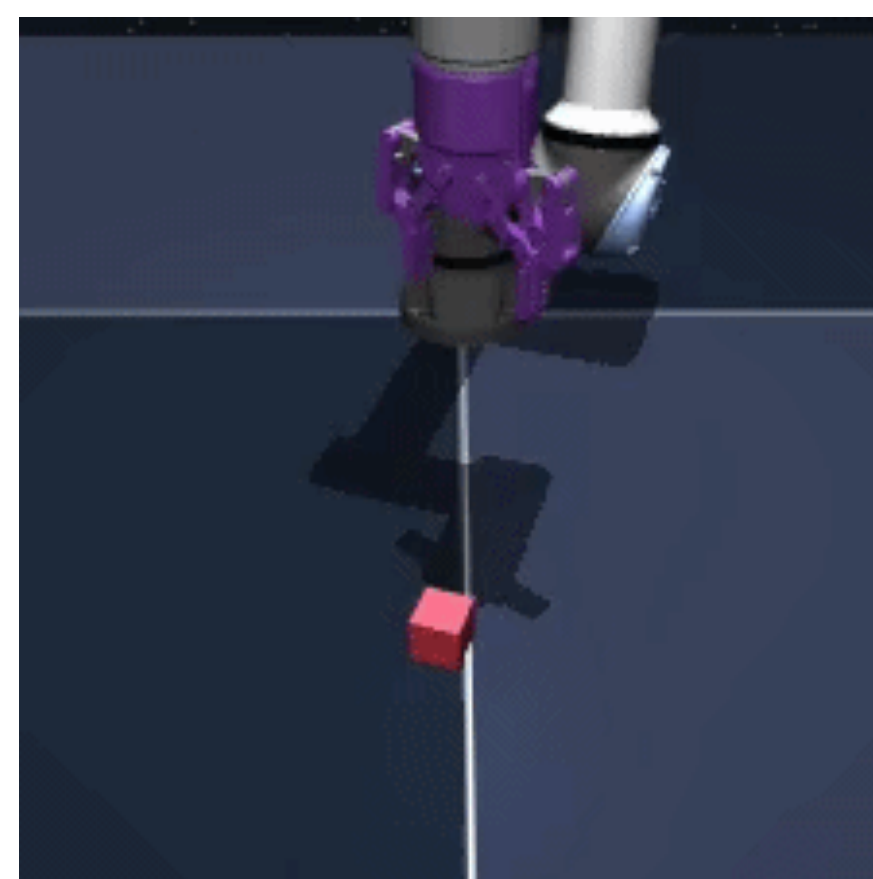
We introduce a sudden environmental change and evaluate the level of “surprise” (prediction error) experienced by the world model.



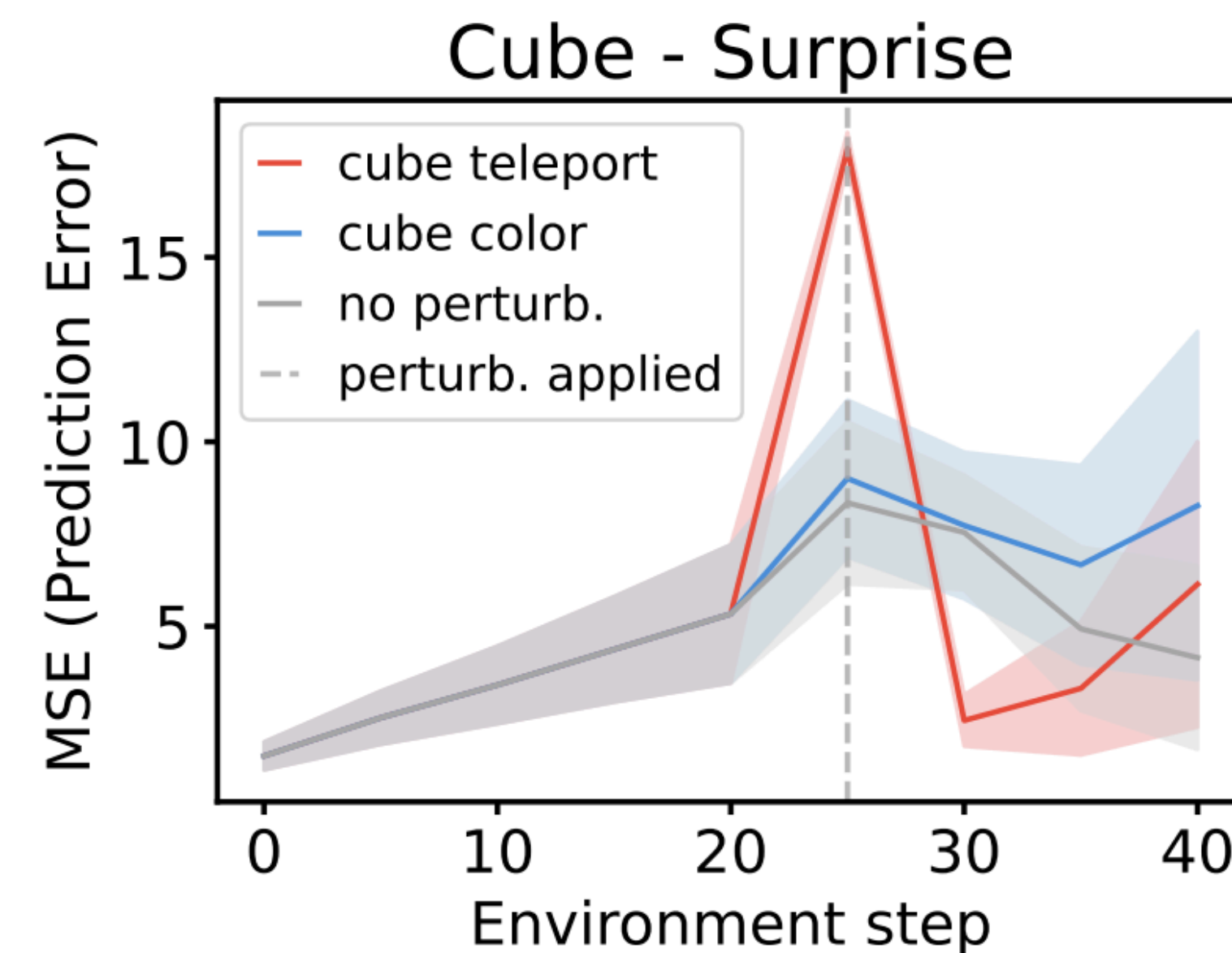
No Perturbation



Cube Color

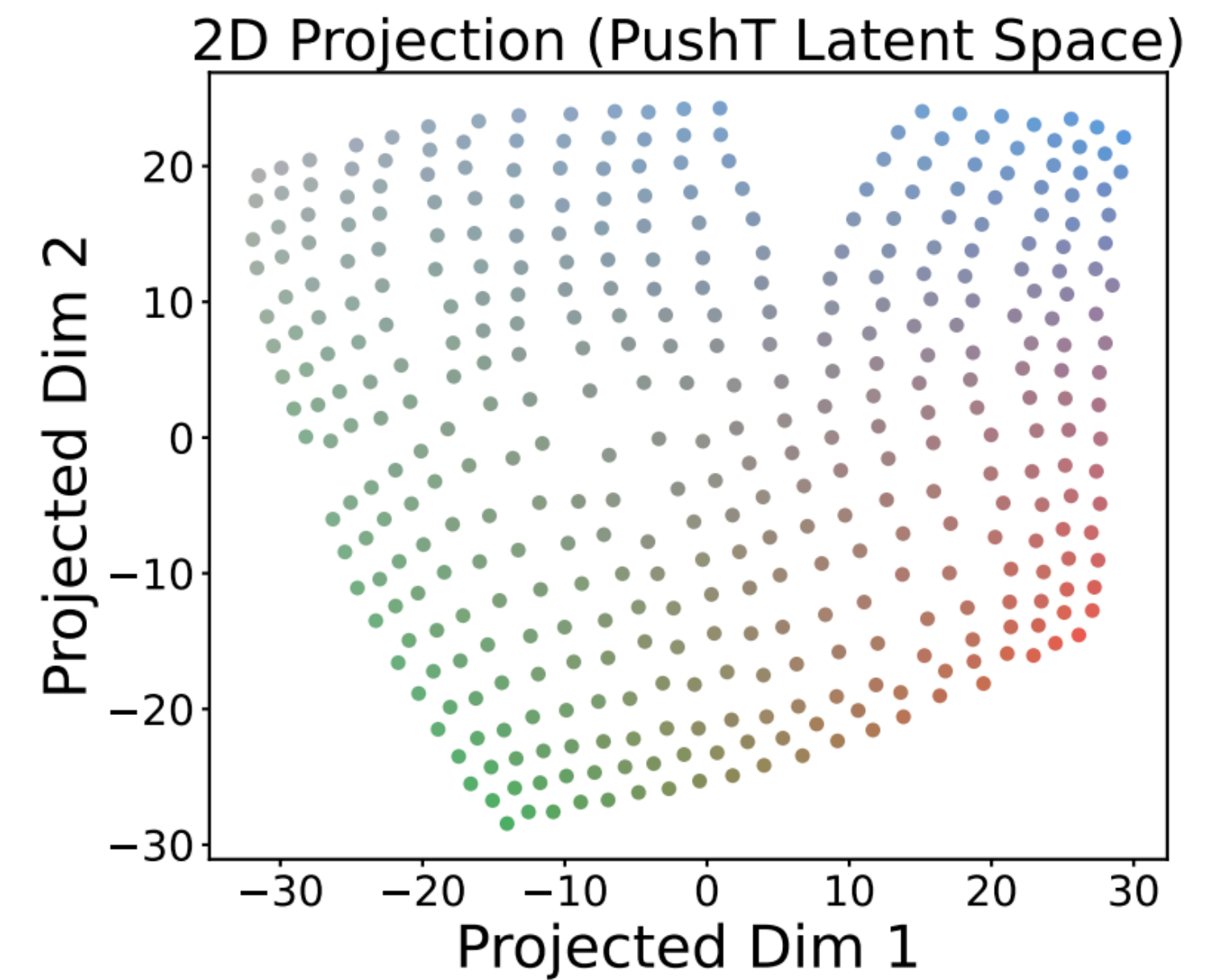
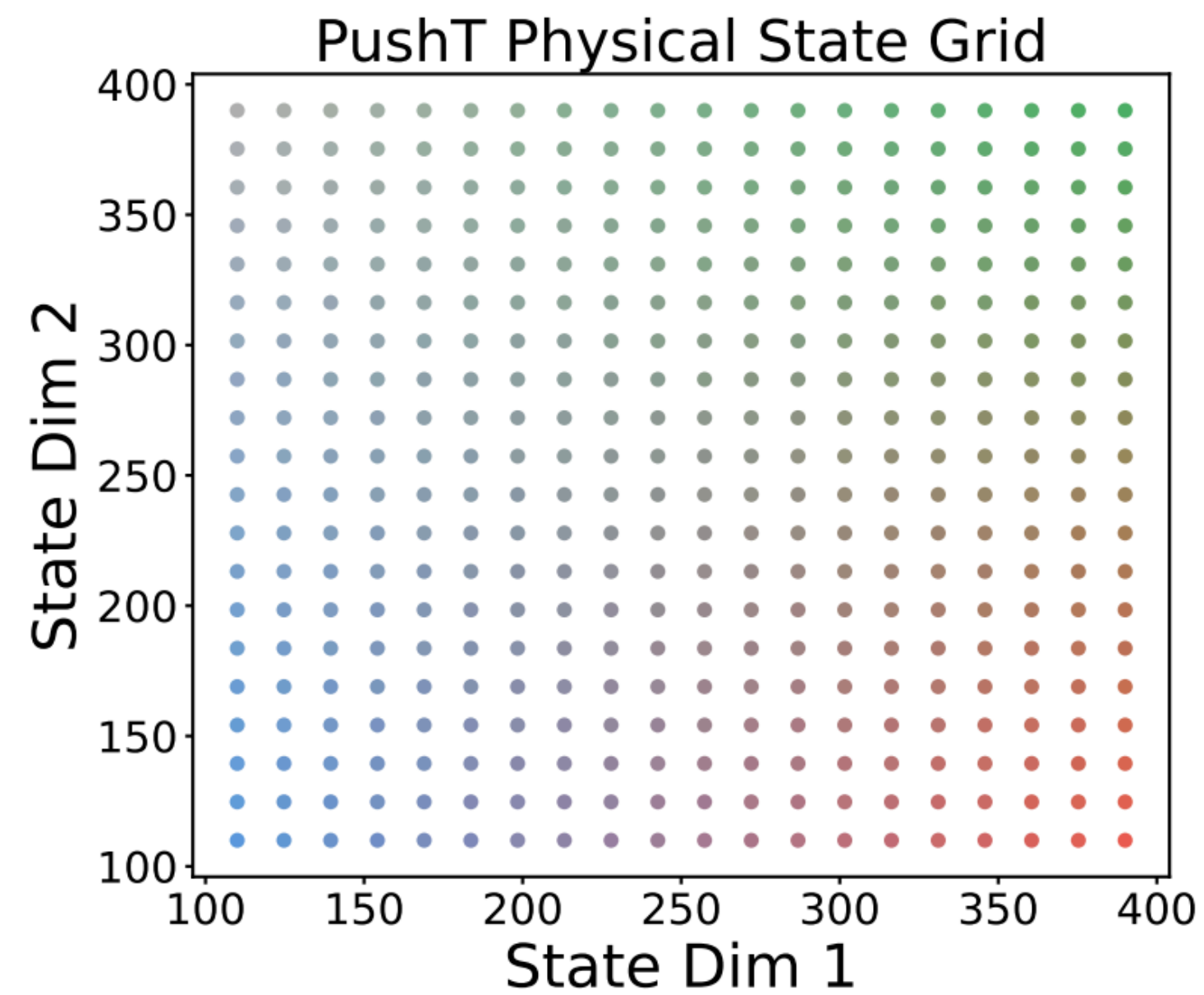


Cube Teleportation

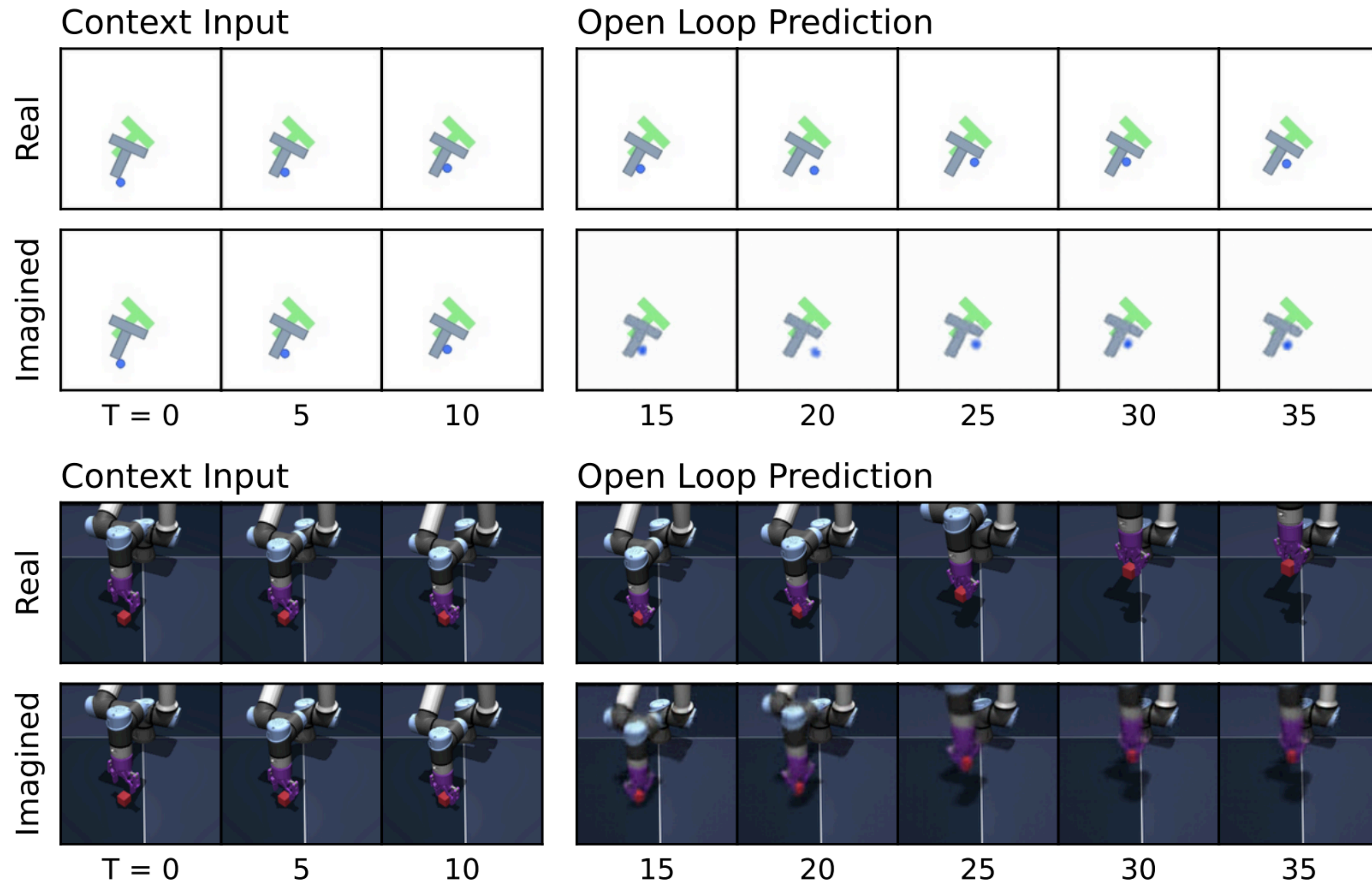


Analyzing Latent Space

Relative distance in latent space



Analyzing Latent Space



Limitations

or research opportunities

- Currently short term horizon planning
- Representation at a single temporal level (need hierarchies)
- “Toy” environment setup (real-world robotics? Minecraft?)
- Un-realistic goal specification

stable-worldmodel

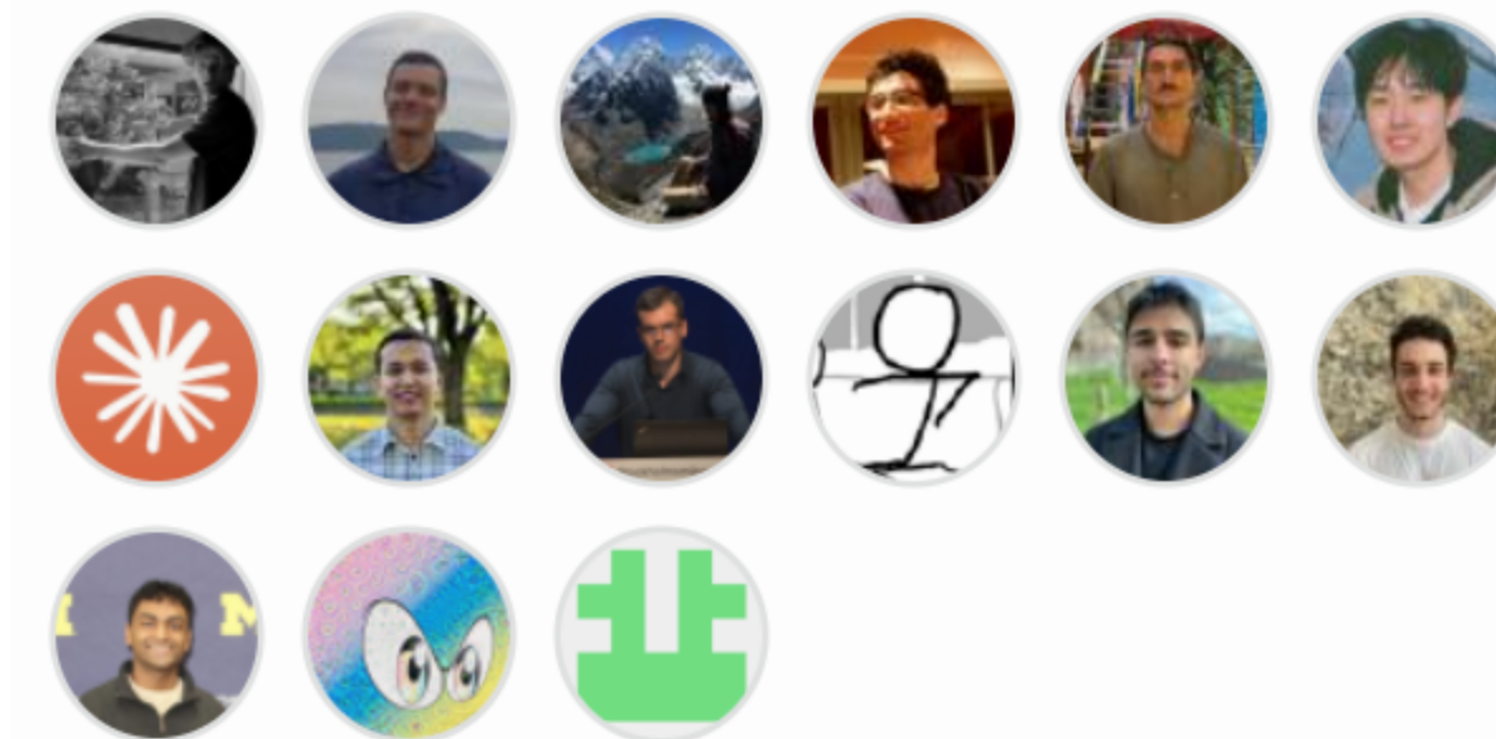
- All baseline (LeWM, PLDM, DINO-WM, IVL, IQL, ...)
- All solver (CEM, MPPI, Adam, ...)
- All environment (Push-T, TwoRoom, OG-Bench, DMC)
- Heavily Tested, documentation available

```
pip install stable-worldmodel
```

 **galilai-group/stable-worldmodel** Public

Reliable, minimal and scalable library for evaluating and conducting world model research

● Python ☆ 395 🔗 53



Thanks!

